Graphs — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

299. Graph and Types

Understanding the Problem NEW

A Graph is the most general connection-based structure covered so far — unlike a Tree (which has one root and no cycles), a graph is simply a set of 'nodes' (vertices) connected by 'edges,' with no restrictions: any node can connect to any other node, cycles are allowed, and edges can be one-directional (Directed Graph) or two-directional (Undirected Graph), and may carry weights (costs) or not.

Brute Force Approach NEW

Nothing to contrast against here — this is foundational vocabulary. It's worth explicitly noting that a Tree is actually just a special, restricted kind of graph (connected, no cycles, exactly one path between any two nodes) — everything learned about trees generalizes, with extra care needed once cycles become possible.

Why We Optimize NEW

Graphs model an enormous range of real-world problems — social networks, maps/road systems, dependency chains, computer networks — which is why this is one of the largest chapters in the book. The biggest conceptual jump from trees: since graphs can have cycles, naive recursive traversal (which worked fine on trees) can now loop forever unless you explicitly track **visited nodes** — a single new piece of bookkeeping that becomes essential in almost every algorithm in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding graph data structure and its types.

Visual Diagram

```
Graph Types:
       1. Undirected Graph:
          A --- B
          |     |
          C --- D
```

```
2. Directed Graph (Digraph):
   A → B
   ↓   ↓
   C → D

3. Weighted Graph:
   A --5-- B
   |       |
   2       3
   |       |
   C --4-- D

4. Cyclic vs Acyclic:
   Cyclic: A → B → C → A
   Acyclic: A → B → C

Graph Terminology:
- Vertices/Nodes: A, B, C, D
- Edges: Connections between vertices
- Degree: Number of edges incident to vertex
- Path: Sequence of vertices
- Cycle: Path that starts and ends at same vertex
```

300-301. Graph Representation

Understanding the Problem NEW

Before writing any graph algorithm, you need a way to actually store the graph in memory. The two standard representations are an **Adjacency Matrix** (an n×n grid where cell [i][j] indicates whether an edge exists between i and j) and an **Adjacency List** (for each node, a list of its directly connected neighbours).

Brute Force Approach NEW

An adjacency matrix is simple to reason about and offers O(1) 'is there an edge between i and j?' lookups, but it always uses O(n²) space regardless of how many edges actually exist — wasteful for 'sparse' graphs (relatively few edges compared to the number of possible pairs).

Why We Optimize NEW

An **Adjacency List** uses O(n + edges) space instead — only storing the connections that actually exist — which is dramatically better for sparse graphs (most real-world graphs are sparse: a social network doesn't have every person connected to every other person). The trade-off is that checking 'is there a specific edge between i and j' becomes O(degree of i) instead of O(1) — but since most graph algorithms need to 'visit all neighbours of a node' far more often than 'check one specific pair,' the adjacency list's efficient neighbour iteration usually wins out, which is why it's the default representation used throughout the rest of this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Different ways to represent graphs.

Visual Diagram

```
Graph:
           0 --- 1
           |     |
           3 --- 2

       Adjacency Matrix (4x4):
```

```
    0 1 2 3
0: 0 1 0 1
1: 1 0 1 0
2: 0 1 0 1
3: 1 0 1 0

Adjacency List:
0 → [1, 3]
1 → [0, 2]
2 → [1, 3]

3 → [0, 2]

Edge List:
[(0,1), (0,3), (1,2), (2,3)]
```

Java Implementation

```
// Adjacency List
class Graph {
    private List<List<Integer>> adjList;
    private int vertices;

    Graph(int vertices) {
        this.vertices = vertices;
        adjList = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    void addEdge(int u, int v) {
        adjList.get(u).add(v);
        adjList.get(v).add(u); // For undirected
    }

    List<Integer> getNeighbors(int v) {
        return adjList.get(v);
    }
}

// Adjacency Matrix
class GraphMatrix {
    private int[][] matrix;
    private int vertices;

    GraphMatrix(int vertices) {
        this.vertices = vertices;
        matrix = new int[vertices][vertices];
    }

    void addEdge(int u, int v) {
```

```
            matrix[u][v] = 1;
            matrix[v][u] = 1; // For undirected
        }
    }
```

Time & Space Complexity

Time Complexity:

302. Connected Components

Understanding the Problem NEW

Find all the 'connected components' of a graph — separate clusters of nodes where every node in a cluster can reach every other node in the same cluster, but nodes in different clusters can't reach each other at all.

Brute Force Approach NEW

You could check every pair of nodes for reachability individually — $O(n^2)$ pairs, each requiring a traversal to check connectivity, giving a very expensive $O(n^3)$ or worse overall.

Why We Optimize NEW

Instead of checking pairs, do a single traversal (BFS or DFS) starting from any unvisited node — this traversal will naturally reach *every* node in that node's entire connected component, all in one go. Mark every visited node, then repeat by starting a new traversal from the next unvisited node (a new component), continuing until every node has been visited. This finds every connected component in a total of $O(V + E)$ — each node and edge is only ever processed once across all the traversals combined, rather than repeatedly testing pairwise reachability.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find connected components in undirected graph.

Visual Diagram

```
Graph:
      Component 1: 0-1-2
      Component 2: 3-4
      Component 3: 5

      Visited array:
      0: visited
      1: visited
      2: visited
      3: visited
```

```
      4: visited
      5: visited

      Components count = 3
```

Solution Approach

Concept: DFS/BFS on unvisited nodes.

Algorithm

```
Algorithm COUNT_COMPONENTS(graph):
          visited = new boolean[V]
          count = 0

          For i = 0 to V-1:
              If !visited[i]:
                  DFS(i, visited)
                  count++
          Return count
```

Java Implementation

```
class Solution {
          public int countComponents(int n, int[][] edges) {
              List<List<Integer>> graph = new ArrayList<>();
              for (int i = 0; i < n; i++) {
                  graph.add(new ArrayList<>());
              }
              for (int[] edge : edges) {
                  graph.get(edge[0]).add(edge[1]);
                  graph.get(edge[1]).add(edge[0]);
              }

                 boolean[] visited = new boolean[n];
                 int count = 0;

                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {
                         dfs(i, graph, visited);
                         count++;
                     }
                 }
                 return count;
          }

          private void dfs(int node, List<List<Integer>> graph, boolean[] visited)
{
              visited[node] = true;
              for (int neighbor : graph.get(node)) {
```

```
                if (!visited[neighbor]) {
                    dfs(neighbor, graph, visited);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

303. BFS (Breadth First Search)

Understanding the Problem NEW

Traverse a graph breadth-first — visiting the starting node, then all of its direct neighbours, then all of *their* unvisited neighbours, and so on, level by level, exactly like Level Order traversal on a tree (problem 251), but now with a graph's added possibility of cycles.

Brute Force Approach NEW

Without tracking visited nodes, a naive traversal could revisit the same nodes repeatedly through a cycle, looping forever — the entire challenge here (unlike the tree version) is correctly handling cycles.

Why We Optimize NEW

Use the same **queue-based approach** as tree Level Order, but add a **visited set**: mark a node visited the moment it's *enqueued* (not when dequeued) to prevent the same node from being added to the queue multiple times through different paths. This guarantees each node and edge is processed exactly once, achieving O(V + E) time — the visited-set is the one new ingredient graphs require that trees didn't need, since a tree's lack of cycles made accidental revisiting structurally impossible.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse graph level by level.

Visual Diagram

```
Graph:
           0 --- 1
           |     |
           3 --- 2

       BFS from 0:
       Queue: [0]
       Visit 0, add neighbors: [1,3]
       Visit 1, add neighbors: [2]
       Visit 3
```

```
       Visit 2

       Order: 0, 1, 3, 2

       BFS Tree:
            0
           / \
         1     3
         |
         2
```

Solution Approach

Concept: Queue for level order traversal.

Algorithm

```
Algorithm BFS(start):
           queue = Queue()
           visited = boolean[V]
           queue.add(start)
           visited[start] = true

           While queue not empty:
               node = queue.poll()
               print(node)
               For neighbor in graph[node]:
                   If !visited[neighbor]:
                       visited[neighbor] = true
                       queue.add(neighbor)
```

Java Implementation

```
class Solution {
           public void bfs(int start, List<List<Integer>> graph) {
               boolean[] visited = new boolean[graph.size()];
               Queue<Integer> queue = new LinkedList<>();
               queue.add(start);
               visited[start] = true;

                 while (!queue.isEmpty()) {
                     int node = queue.poll();
                     System.out.print(node + " ");

                     for (int neighbor : graph.get(node)) {

                      if (!visited[neighbor]) {
                          visited[neighbor] = true;
                          queue.add(neighbor);
                      }
```

```
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

304. DFS (Depth First Search)

Understanding the Problem NEW

Traverse a graph depth-first — go as deep as possible down one path before backtracking, similar to the recursive Preorder traversal idea from trees (problem 248), but again needing to handle cycles.

Brute Force Approach NEW

Same risk as BFS without care: naive recursion could loop forever through a cycle without a way to detect 'I've already been here.'

Why We Optimize NEW

Use the same **recursive (or explicit-stack) approach** as tree preorder, adding a **visited set** exactly like BFS does: mark each node visited as soon as you arrive, and only recurse into neighbours that haven't been visited yet. This achieves O(V + E) time just like BFS — the key structural choice between BFS and DFS is queue vs. stack/recursion, but the 'add a visited set to handle cycles' fix is identical for both, and this same DFS-with-visited-tracking template becomes the backbone for a large fraction of the remaining problems in this chapter (cycle detection, topological sort, bridges, articulation points).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse graph using recursion/stack.

Visual Diagram

```
Graph:
           0 --- 1
           |     |
           3 --- 2

       DFS from 0:
       Visit 0
       Visit neighbor 1
       Visit neighbor 2
       Visit neighbor 3
       Backtrack
```

```
       Order: 0, 1, 2, 3

       DFS Tree:
           0
           |
           1
           |
           2
           |
           3
```

Solution Approach

Concept: Stack/Recursion for depth traversal.

Algorithm

```
Algorithm DFS(node, visited):
           visited[node] = true
           print(node)
           For neighbor in graph[node]:
               If !visited[neighbor]:
                   DFS(neighbor, visited)
```

Java Implementation

```
class Solution {
           // Recursive DFS
           public void dfs(int node, List<List<Integer>> graph, boolean[] visited)
{
               visited[node] = true;
               System.out.print(node + " ");

                 for (int neighbor : graph.get(node)) {
                     if (!visited[neighbor]) {
                         dfs(neighbor, graph, visited);
                     }
                 }
           }

           // Iterative DFS
           public void dfsIterative(int start, List<List<Integer>> graph) {
               boolean[] visited = new boolean[graph.size()];
               Stack<Integer> stack = new Stack<>();

              stack.push(start);

              while (!stack.isEmpty()) {
                  int node = stack.pop();
```

```
                if (!visited[node]) {
                    visited[node] = true;
                    System.out.print(node + " ");

                    for (int neighbor : graph.get(node)) {
                        if (!visited[neighbor]) {
                            stack.push(neighbor);
                        }
                    }
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

305. Number of Provinces

Understanding the Problem NEW

Given an adjacency matrix representing connections between 'provinces' (cities directly connected if 1, else 0), count how many separate provinces (connected groups) exist — a direct real-world framing of Connected Components (302).

Brute Force Approach NEW

Check every pair of cities for a connecting path individually — $O(n^2)$ to $O(n^3)$.

Why We Optimize NEW

Exactly the same technique as problem 302: run DFS or BFS from each unvisited city, marking every city reachable from it as part of the same province, and count how many separate traversals were needed. Recognizing that 'count provinces' is literally 'count connected components' with different words is the main insight — no new algorithm required.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count connected components in graph (provinces).

Visual Diagram

```
Matrix:
       [1,1,0]
       [1,1,0]
       [0,0,1]

       Province 1: [0,1]
       Province 2: [2]
       Count = 2

       DFS from 0: visits 0,1
       DFS from 2: visits 2
```

Solution Approach

Concept: DFS on adjacency matrix.

Algorithm

```
Algorithm FIND_PROVINCES(isConnected):
           n = isConnected.length
           visited = new boolean[n]
           count = 0

           For i = 0 to n-1:
               If !visited[i]:
                   dfs(i, isConnected, visited)
                   count++
           Return count
```

Java Implementation

```
class Solution {
           public int findCircleNum(int[][] isConnected) {
               int n = isConnected.length;
               boolean[] visited = new boolean[n];
               int count = 0;

                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {
                         dfs(i, isConnected, visited);
                         count++;
                     }
                 }
                 return count;
           }

           private void dfs(int node, int[][] isConnected, boolean[] visited) {
               visited[node] = true;
               for (int i = 0; i < isConnected.length; i++) {
                   if (isConnected[node][i] == 1 && !visited[i]) {
                       dfs(i, isConnected, visited);
                   }

               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

306. Connected Components in Matrix

Understanding the Problem NEW

Given a 2D binary grid (representing land 'connected component regions'), find the connected components — but now 'neighbours' means adjacent grid cells (up/down/left/right), not an explicit edge list.

Brute Force Approach NEW

You could try to convert the grid into an explicit graph (with nodes for each cell and edges to adjacent cells) and then run standard connected-components logic — this works but adds unnecessary conversion overhead.

Why We Optimize NEW

Treat the grid **directly** as an implicit graph: 'neighbours' of a cell are simply its up/down/left/right adjacent cells (computed via simple index arithmetic, no explicit edge list needed), and run DFS/BFS exactly as in problem 302, marking visited cells directly in the grid (or a separate visited grid). This is the first of several grid-based graph problems in this chapter — recognizing that a grid *is* a graph in disguise (with implicit adjacency via neighbouring indices) avoids the overhead of explicitly building a graph structure for something whose connectivity is already implied by position.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find connected components in a binary matrix.

Visual Diagram

```
Matrix:
       1 1 0
       0 1 0
       0 0 1

       Components: 2
       Component 1: (0,0), (0,1), (1,1)
       Component 2: (2,2)
```

```
       DFS: Visit all connected 1s
```

Solution Approach

Concept: DFS on grid with 4 directions.

Algorithm

```
Algorithm COUNT_COMPONENTS_MATRIX(grid):
           m = grid.length, n = grid[0].length
           visited = boolean[m][n]
           count = 0

           For i = 0 to m-1:
               For j = 0 to n-1:
                   If grid[i][j] == 1 and !visited[i][j]:
                        dfs(i, j, grid, visited)
                        count++
           Return count
```

Java Implementation

```
class Solution {
           public int countComponents(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];
               int count = 0;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1 && !visited[i][j]) {
                             dfs(i, j, grid, visited);
                             count++;
                         }
                     }
                 }
                 return count;
           }

           private void dfs(int i, int j, int[][] grid, boolean[][] visited) {
               int m = grid.length, n = grid[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0 ||
visited[i][j]) {
                   return;
               }

                 visited[i][j] = true;

              int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
```

```
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], grid, visited);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

307. Rotten Oranges

Understanding the Problem NEW

Given a grid where cells are empty, contain a fresh orange, or a rotten orange, and rot spreads to adjacent fresh oranges each minute, find the minimum time for all oranges to rot (or -1 if impossible).

Brute Force Approach NEW

Simulate minute by minute, scanning the *entire* grid on every single minute to find which fresh oranges are now adjacent to a rotten one — O(minutes × grid size), redundantly rescanning the whole grid repeatedly even though most cells aren't relevant on most minutes.

Why We Optimize NEW

This calls for **Multi-Source BFS**: instead of starting BFS from just one node, start it simultaneously from *every* already-rotten orange at once (enqueue them all before beginning), then proceed with standard BFS — since BFS naturally processes level by level, and each level here corresponds exactly to one minute passing, this correctly simulates simultaneous spreading from multiple sources without redundant rescanning. This achieves O(rows × cols) total, visiting each cell once, instead of repeatedly rescanning the whole grid every single minute — 'multi-source BFS' (seed the queue with several starting points before the traversal begins) is a small but powerful variation on ordinary BFS that reappears in problem 311.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Time to rot all oranges in a grid.

Visual Diagram

```
Grid:
       [2,1,1]
       [1,1,0]
       [0,1,1]

       Time 0: (0,0) is rotten
```

```
       Time 1: (0,1), (1,0) rot
       Time 2: (0,2), (1,1), (2,1) rot
       Time 3: (2,2) rot
       Output: 3

       BFS with multiple sources:
       Queue all rotten oranges
       Level by level traversal
```

Solution Approach

Concept: Multi-source BFS.

Algorithm

```
Algorithm ROTTEN_ORANGES(grid):
           queue = Queue()
           fresh = 0
           time = 0

            For each cell:
                If grid[i][j] == 2: queue.add((i,j))
                If grid[i][j] == 1: fresh++

            While queue not empty and fresh > 0:
                size = queue.size()
                For each in queue:
                    Try all 4 directions
                    If neighbor is fresh:
                        grid[neighbor] = 2
                        queue.add(neighbor)
                        fresh--
                time++

            Return fresh == 0 ? time : -1
```

Java Implementation

```
class Solution {
           public int orangesRotting(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               Queue<int[]> queue = new LinkedList<>();
               int fresh = 0;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 2) {
                             queue.add(new int[]{i, j});
                         } else if (grid[i][j] == 1) {
```

```
                        fresh++;
                    }
                }
            }

            if (fresh == 0) return 0;
            int time = 0;
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

            while (!queue.isEmpty() && fresh > 0) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    int[] cell = queue.poll();
                    for (int[] dir : dirs) {
                        int x = cell[0] + dir[0];
                        int y = cell[1] + dir[1];
                        if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] ==
1) {
                            grid[x][y] = 2;
                            queue.add(new int[]{x, y});
                            fresh--;
                        }
                    }
                }
                time++;
            }
            return fresh == 0 ? time : -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

308. Flood Fill

Understanding the Problem NEW

Given a grid (like an image) and a starting pixel, change the color of that pixel and every pixel connected to it (via matching color, adjacent cells) to a new color — the 'paint bucket' tool in image editors.

Brute Force Approach NEW

There's no slower alternative worth contrasting — this is essentially Connected Components (302/306) applied directly, where the 'connection' is 'adjacent cell with the same original color.'

Why We Optimize NEW

Run DFS (or BFS) from the starting pixel, only recursing into neighbouring cells that share the *original* color (checked before it gets overwritten, to avoid immediately re-matching the new color and causing infinite recursion if old and new colors happen to be checked incorrectly), changing each visited cell's color as you go. This is a direct, minimal-twist application of grid-based connected-component traversal (306) — the 'new insight' here is really just recognizing that a common image-editing tool is exactly a graph traversal problem in disguise.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Fill connected region with new color.

Visual Diagram

```
Image:
       [1,1,1]
       [1,1,0]
       [1,0,1]
       Start: (1,1), NewColor: 2

       Result:
       [2,2,2]
       [2,2,0]
       [2,0,1]
```

```
       DFS: Replace all connected 1s with 2
```

Solution Approach

Concept: DFS with color replacement.

Algorithm

```
Algorithm FLOOD_FILL(image, sr, sc, newColor):
           if image[sr][sc] == newColor: return image
           oldColor = image[sr][sc]
           dfs(image, sr, sc, oldColor, newColor)
           return image

       Algorithm dfs(image, i, j, oldColor, newColor):
           if out of bounds or image[i][j] != oldColor: return
           image[i][j] = newColor
           dfs all 4 directions
```

Java Implementation

```
class Solution {
           public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
               int oldColor = image[sr][sc];
               if (oldColor == newColor) return image;
               dfs(image, sr, sc, oldColor, newColor);
               return image;
           }

           private void dfs(int[][] image, int i, int j, int oldColor, int
newColor) {
               int m = image.length, n = image[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || image[i][j] != oldColor) {
                   return;
               }

                 image[i][j] = newColor;
                 int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
                 for (int[] dir : dirs) {
                     dfs(image, i + dir[0], j + dir[1], oldColor, newColor);
                 }
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

309-310. Cycle Detection in Undirected Graph

Understanding the Problem NEW

Detect whether an undirected graph contains a cycle (a path that starts and ends at the same node without reusing any edge).

Brute Force Approach NEW

You could try to enumerate every possible path and check if any revisits a node — exponential, since the number of possible paths in a graph can be very large.

Why We Optimize NEW

During a DFS traversal, if you ever encounter an **already-visited node that isn't the node you just came from** (i.e., not simply your immediate parent in the traversal), you've found a cycle — an edge going back to some earlier-visited node other than your direct predecessor. This requires **tracking the parent** of each DFS call explicitly (since in an undirected graph, the edge back to your immediate parent is expected and not a cycle — you arrived via that edge, so seeing it again isn't a 'true' new cycle). This detects a cycle in a single O(V+E) DFS traversal, instead of exponentially enumerating paths — this 'track parent, flag a cycle if you see a visited non-parent node' technique is specific to *undirected* graphs; directed graphs need a different approach (three-state coloring, problem 318), since a directed edge's 'reverse' direction isn't automatically expected the way it is here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect cycles in undirected graph.

Visual Diagram

```
Graph with cycle:
       0 --- 1
       |     |
       3 --- 2

       DFS from 0:
       Visit 0
```

```
Visit 1
Visit 2
Visit 3 → neighbor 0 already visited and not parent → cycle

Graph without cycle:
0 --- 1
|     |
3     2

DFS from 0:
Visit 0
Visit 1
Visit 2
Backtrack
Visit 3 → no cycle
```

Solution Approach

Concept: DFS with parent tracking.

Algorithm (DFS)

Algorithm HAS_CYCLE_DFS(graph):

visited = boolean[V]

For i = 0 to V-1:

If !visited[i]:

If dfs(i, -1, visited): Return true

Return false

Algorithm dfs(node, parent, visited):

visited[node] = true

For neighbor in graph[node]:

If !visited[neighbor]:

If dfs(neighbor, node, visited): Return true

Else If neighbor != parent:

Return true

Return false

Java Implementation

```
class Solution {
           public boolean hasCycle(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);

                  graph.get(edge[1]).add(edge[0]);
              }

              boolean[] visited = new boolean[n];
              for (int i = 0; i < n; i++) {
                  if (!visited[i]) {
                      if (dfs(i, -1, graph, visited)) {
                          return true;
                      }
                  }
              }
              return false;
          }

          private boolean dfs(int node, int parent, List<List<Integer>> graph,
boolean[] visited) {
              visited[node] = true;
              for (int neighbor : graph.get(node)) {
                  if (!visited[neighbor]) {
                      if (dfs(neighbor, node, graph, visited)) {
                          return true;
                      }
                  } else if (neighbor != parent) {
                      return true;
                  }
              }
              return false;
          }
      }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

311. 0/1 Matrix

Understanding the Problem NEW

Given a binary matrix, for every cell find the distance to the nearest 0 (using standard grid adjacency, one step per adjacent cell).

Brute Force Approach NEW

For each cell containing a 1, run a fresh BFS (or scan outward) to find the nearest 0 — O(cells) BFS calls, each potentially O(cells), giving O(cells²) overall — heavily redundant since many searches will re-explore overlapping territory.

Why We Optimize NEW

Exactly the **Multi-Source BFS** technique from Rotten Oranges (307): seed the BFS queue with *every* cell that's already a 0 (distance 0) simultaneously, then expand outward level by level — each cell's distance is naturally determined by which BFS 'wave' first reaches it. This computes every cell's distance in a single O(rows × cols) pass, instead of O(cells²) from re-running BFS individually from every 1-cell — reinforcing multi-source BFS as a reusable technique whenever you need 'distance to nearest occurrence of X' across an entire grid at once, rather than per-cell searches.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find distance of each cell to nearest 0.

Visual Diagram

```
Matrix:
       [0,0,0]
       [0,1,0]
       [1,1,1]

       Result:
       [0,0,0]
       [0,1,0]
       [1,2,1]

       BFS from all zeros:
```

```
        Add all 0s to queue
        BFS level by level
        Distance = level
```

Solution Approach

Concept: Multi-source BFS.

Algorithm

```
Algorithm UPDATE_MATRIX(matrix):
            m = matrix.length, n = matrix[0].length
            dist = new int[m][n]
            queue = Queue()

            For each cell:
                If matrix[i][j] == 0:
                    dist[i][j] = 0
                    queue.add((i,j))
                Else:
                    dist[i][j] = INF

            BFS and update distances
            Return dist
```

Java Implementation

```
class Solution {
            public int[][] updateMatrix(int[][] mat) {
                int m = mat.length, n = mat[0].length;
                int[][] dist = new int[m][n];
                Queue<int[]> queue = new LinkedList<>();

                  for (int i = 0; i < m; i++) {
                      for (int j = 0; j < n; j++) {
                          if (mat[i][j] == 0) {
                              dist[i][j] = 0;
                              queue.add(new int[]{i, j});
                          } else {
                              dist[i][j] = Integer.MAX_VALUE;
                          }
                      }
                  }
                  int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

               while (!queue.isEmpty()) {
                   int[] cell = queue.poll();
                   for (int[] dir : dirs) {
                       int x = cell[0] + dir[0];
                       int y = cell[1] + dir[1];
```

```
                    if (x >= 0 && x < m && y >= 0 && y < n) {
                        if (dist[x][y] > dist[cell[0]][cell[1]] + 1) {
                            dist[x][y] = dist[cell[0]][cell[1]] + 1;
                            queue.add(new int[]{x, y});
                        }
                    }
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

312. Surrounded Regions

Understanding the Problem NEW

Given a grid of 'X' and 'O' characters, capture (flip to 'X') every region of 'O's that is fully surrounded by 'X's — but regions of 'O' touching the grid's border should NOT be captured, since they're not actually 'surrounded.'

Brute Force Approach NEW

You might try to check, for each 'O' region, whether it's surrounded by scanning its entire boundary and checking every adjacent cell — awkward to get right directly, especially distinguishing 'touches the border' from 'surrounded' cleanly.

Why We Optimize NEW

Flip the perspective: instead of trying to detect 'surrounded' regions directly, **first mark every 'O' region that touches the border as safe** (run DFS/BFS starting from every border cell that's an 'O', marking everything reachable from there) — since anything connected to the border obviously can't be fully surrounded. Afterward, in a single pass, flip every remaining *unmarked* 'O' to 'X' (these are exactly the truly surrounded ones) and restore the marked (border-connected) ones back to 'O'. This 'identify the safe/exempt cells first via traversal from the boundary, then handle everything else' strategy avoids ambiguous case-by-case surrounded-region detection, and reappears almost identically in problem 313.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Capture surrounded regions (flip 'O' to 'X').

Visual Diagram

```
Board:
       X X X X
       X 0 0 X
       X X 0 X
       X 0 X X

       Result:
```

```
X X X X
X X X X
X X X X
X O X X

Approach:
Mark boundary 'O' and connected 'O' as safe
Flip all other 'O' to 'X'
```

Solution Approach

Concept: DFS from boundaries.

Algorithm

```
Algorithm SURROUNDED_REGIONS(board):
           m = board.length, n = board[0].length

            // Mark boundary 'O's
            For each boundary cell:
                If board[i][j] == 'O':
                    dfs(i, j, board)

            // Flip remaining 'O' to 'X'
            // Flip marked to 'O'
```

Java Implementation

```
class Solution {
           public void solve(char[][] board) {
               int m = board.length, n = board[0].length;

                 // Mark boundary O's and connected ones
                 for (int i = 0; i < m; i++) {
                     if (board[i][0] == 'O') dfs(i, 0, board);
                     if (board[i][n-1] == 'O') dfs(i, n-1, board);
                 }
                 for (int j = 0; j < n; j++) {
                     if (board[0][j] == 'O') dfs(0, j, board);
                     if (board[m-1][j] == 'O') dfs(m-1, j, board);
                 }

                 // Flip remaining
                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (board[i][j] == 'O') board[i][j] = 'X';
                         if (board[i][j] == '#') board[i][j] = 'O';
                     }

             }
```

```
        }

        private void dfs(int i, int j, char[][] board) {
            int m = board.length, n = board[0].length;
            if (i < 0 || i >= m || j < 0 || j >= n || board[i][j] != 'O') return;

            board[i][j] = '#';
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], board);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

313. Number of Enclaves

Understanding the Problem NEW

Count the number of land cells in a grid that *cannot* reach the grid's boundary by moving through adjacent land cells — the complement of 'land connected to the border.'

Brute Force Approach NEW

For every land cell, check if there's a path to the boundary — O(cells) searches, each potentially O(cells), redundant and expensive.

Why We Optimize NEW

Exactly the same 'mark the boundary-connected cells first, then count what's left' strategy as Surrounded Regions (312): run DFS/BFS from every boundary land cell, marking everything reachable; then simply count land cells that were *never*

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count land cells that cannot reach boundary.

Visual Diagram

```
Grid:
       0 0 0 0
       1 0 1 0
       0 1 1 0
       0 0 0 0

       Enclaves: 3 (cells at (1,2), (2,1), (2,2))

       Approach:
       Mark all land connected to boundary
       Count remaining land cells
```

Solution Approach

Concept: DFS from boundary.

Algorithm

```
Algorithm ENCLAVES(grid):
           m = grid.length, n = grid[0].length
           visited = boolean[m][n]

           // DFS from boundary land
           For each boundary:
               If grid[i][j] == 1:
                   dfs(i, j, grid, visited)

           // Count unvisited land
           count = 0
           For each cell:
               If grid[i][j] == 1 and !visited[i][j]:
                   count++
           Return count
```

Java Implementation

```
class Solution {
           public int numEnclaves(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];

                 // Mark boundary connected land
                 for (int i = 0; i < m; i++) {
                     if (grid[i][0] == 1) dfs(i, 0, grid, visited);
                     if (grid[i][n-1] == 1) dfs(i, n-1, grid, visited);
                 }
                 for (int j = 0; j < n; j++) {
                     if (grid[0][j] == 1) dfs(0, j, grid, visited);
                     if (grid[m-1][j] == 1) dfs(m-1, j, grid, visited);
                 }

                 // Count enclaves
                 int count = 0;
                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {

                      if (grid[i][j] == 1 && !visited[i][j]) {
                          count++;
                      }
                    }
               }
               return count;
           }

           private void dfs(int i, int j, int[][] grid, boolean[][] visited) {
               int m = grid.length, n = grid[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0 ||
```

```
visited[i][j]) {
                return;
            }
            visited[i][j] = true;
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], grid, visited);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

314. Word Ladder - 1

Understanding the Problem NEW

Given a start word and end word, and a dictionary of valid words, find the length of the shortest sequence of single-letter-changed words (each step must be a valid dictionary word) connecting start to end.

Brute Force Approach NEW

Try every possible sequence of word transformations via exhaustive search (trying every position/letter change combination at each step) — exponential, since the number of possible transformation paths grows very quickly.

Why We Optimize NEW

Treat this as a **graph where each word is a node**, and an edge exists between two words if they differ by exactly one letter — then the shortest transformation sequence is exactly the **shortest path** in this implicit graph, found via ordinary BFS from the start word (since BFS naturally finds shortest paths in unweighted graphs, as established in problem 303/326). Rather than building the whole word-graph explicitly upfront (which could be expensive), you generate a word's 'neighbours' on the fly by trying every possible single-letter substitution at every position and checking if the result is in the dictionary — achieving roughly O(word length × alphabet size × number of words) instead of an exponential path search, by recognizing 'shortest transformation sequence' as literally a shortest-path-in-an-unweighted-graph problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Shortest transformation sequence from beginWord to endWord.

Visual Diagram

```
beginWord = "hit", endWord = "cog"
       wordList = ["hot","dot","dog","lot","log","cog"]

       hit → hot → dot → dog → cog
       Length = 5
```

```
       BFS:
       Level 0: hit
       Level 1: hot
       Level 2: dot, lot
       Level 3: dog, log
       Level 4: cog
```

Solution Approach

Concept: BFS with word transformations.

Algorithm

```
Algorithm LADDER_LENGTH(beginWord, endWord, wordList):
           wordSet = Set(wordList)
           queue = Queue()
           queue.add(beginWord)
           level = 1

           While queue not empty:
               size = queue.size()
               For i = 0 to size-1:
                   word = queue.poll()
                   If word == endWord: Return level
                   For each char in word:
                       For c in 'a' to 'z':
                           next = word with char replaced
                           If wordSet.contains(next):
                               wordSet.remove(next)
                               queue.add(next)
               level++

           Return 0
```

Java Implementation

```
class Solution {
           public int ladderLength(String beginWord, String endWord, List<String>
wordList) {
               Set<String> wordSet = new HashSet<>(wordList);
               if (!wordSet.contains(endWord)) return 0;

                 Queue<String> queue = new LinkedList<>();
                 queue.add(beginWord);
                 int level = 1;

                 while (!queue.isEmpty()) {
                     int size = queue.size();
                     for (int i = 0; i < size; i++) {
                         String word = queue.poll();
```

```
                    if (word.equals(endWord)) return level;

                    char[] chars = word.toCharArray();
                    for (int j = 0; j < chars.length; j++) {
                        char original = chars[j];
                        for (char c = 'a'; c <= 'z'; c++) {
                            chars[j] = c;
                            String next = new String(chars);
                            if (wordSet.contains(next)) {
                                wordSet.remove(next);
                                queue.add(next);
                            }
                        }
                        chars[j] = original;
                    }
                }
                level++;
            }
            return 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * L * 26) | Space Complexity: O(n)

315. Word Ladder - 2

Understanding the Problem NEW

Same as Word Ladder-1, but now find *every* shortest transformation sequence (all paths of the minimum length), not just the length of one.

Brute Force Approach NEW

Explore every possible transformation path exhaustively and filter for the ones matching the minimum length found — extremely expensive without first knowing what that minimum length even is.

Why We Optimize NEW

Do this in **two phases**: first, run BFS exactly as in Word Ladder-1 to establish each word's shortest distance from the start (not stopping at just one path, but recording every word's level); then, do a **second pass (DFS or backward reconstruction)** that only follows edges from a word at distance `d` to a word at distance `d+1` — reconstructing every path that strictly increases in BFS-level at each step. This 'BFS to establish distances, then DFS restricted to distance-respecting edges to enumerate all paths' two-phase approach avoids exploring paths that couldn't possibly be shortest, which a blind full-path exploration would waste enormous time on.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all shortest transformation sequences.

Visual Diagram

```
beginWord = "hit", endWord = "cog"
       wordList = ["hot","dot","dog","lot","log","cog"]

       Paths:
       hit → hot → dot → dog → cog
       hit → hot → lot → log → cog

       BFS + DFS:
```

```
        Build parent map during BFS
        DFS to construct paths
```

Solution Approach

Concept: BFS for distances, DFS for paths.

Algorithm

```
Algorithm LADDER_PATHS(beginWord, endWord, wordList):
            Build adjacency list
            BFS to find distances
            DFS from endWord to beginWord
            Return all paths
```

Java Implementation

```
class Solution {
            public List<List<String>> findLadders(String beginWord, String endWord,
List<String> wordLi
                Set<String> wordSet = new HashSet<>(wordList);
                List<List<String>> result = new ArrayList<>();
                if (!wordSet.contains(endWord)) return result;

                  Map<String, List<String>> adj = new HashMap<>();
                  Map<String, Integer> dist = new HashMap<>();

                  // BFS
                  Queue<String> queue = new LinkedList<>();
                  queue.add(beginWord);
                  dist.put(beginWord, 0);

                  while (!queue.isEmpty()) {
                      String word = queue.poll();
                      char[] chars = word.toCharArray();
                      for (int i = 0; i < chars.length; i++) {
                          char original = chars[i];
                          for (char c = 'a'; c <= 'z'; c++) {
                              chars[i] = c;
                              String next = new String(chars);
                              if (wordSet.contains(next) && !next.equals(word)) {
                                  adj.computeIfAbsent(word, k -> new
ArrayList<>()).add(next);
                                  if (!dist.containsKey(next)) {
                                      dist.put(next, dist.get(word) + 1);
                                      queue.add(next);
                                  }
                              }
                          }
```

```
                    chars[i] = original;
                }
            }

            // DFS to build paths
            List<String> path = new ArrayList<>();
            path.add(beginWord);
            dfs(beginWord, endWord, dist, adj, path, result);
            return result;
        }

        private void dfs(String word, String endWord, Map<String, Integer> dist,
                         Map<String, List<String>> adj, List<String> path,
                         List<List<String>> result) {
            if (word.equals(endWord)) {
                result.add(new ArrayList<>(path));
                return;
            }

            for (String next : adj.getOrDefault(word, new ArrayList<>())) {
                if (dist.containsKey(next) && dist.get(next) == dist.get(word) +
1) {
                    path.add(next);
                    dfs(next, endWord, dist, adj, path, result);
                    path.remove(path.size() - 1);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * L * 26) | Space Complexity: O(n * paths)

316. Number of Distinct Islands

Understanding the Problem NEW

Given a grid of land ('1') and water ('0'), count how many *distinct shapes* of islands exist — two islands with the exact same shape (even if in different grid positions) count as the same shape, only counted once.

Brute Force Approach NEW

You could compare every pair of islands' full cell coordinate sets to check if they're the same shape (accounting for translation) — O(islands²) comparisons, each potentially expensive.

Why We Optimize NEW

During DFS/BFS traversal of each island, record the **relative path/shape** taken (e.g. the sequence of directions moved from the island's starting cell, or each cell's position relative to the island's top-left corner) rather than absolute coordinates — this creates a canonical 'shape signature' that's identical for two islands with the same shape regardless of where they sit in the grid. Store these signatures in a **hash set** and count distinct entries. This turns an O(islands²) pairwise shape-comparison problem into an O(cells) traversal-with-signature-recording problem, since a hash set naturally deduplicates matching signatures in O(1) average per insertion.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count distinct island shapes.

Visual Diagram

```
Grid:
       1 1 0 1 1
       1 0 0 1 1
       0 0 0 0 0

       Island 1: (0,0),(0,1),(1,0) → shape: 1,1,1
       Island 2: (0,3),(0,4),(1,3),(1,4) → shape: 1,1,1,1
```

```
       Distinct islands: 2

       Normalize shape by subtracting base coordinates
```

Solution Approach

Concept: DFS with shape normalization.

Algorithm

```
Algorithm COUNT_DISTINCT_ISLANDS(grid):
           visited = boolean[m][n]
           islands = HashSet()

           For each cell:
               If grid[i][j] == 1 and !visited[i][j]:
                   shape = []
                   dfs(i, j, i, j, shape, visited, grid)
                   islands.add(shape)

           Return islands.size()
```

Java Implementation

```
class Solution {
           public int countDistinctIslands(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];
               Set<List<String>> islands = new HashSet<>();

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1 && !visited[i][j]) {
                             List<String> shape = new ArrayList<>();
                             dfs(i, j, i, j, grid, visited, shape);
                             islands.add(shape);
                         }
                     }
                 }
                 return islands.size();
           }

           private void dfs(int i, int j, int baseI, int baseJ, int[][] grid,
                            boolean[][] visited, List<String> shape) {
               int m = grid.length, n = grid[0].length;
               if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0 ||
visited[i][j]) {
                   return;

              }
```

```
            visited[i][j] = true;
            shape.add((i - baseI) + "," + (j - baseJ));
            int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
            for (int[] dir : dirs) {
                dfs(i + dir[0], j + dir[1], baseI, baseJ, grid, visited, shape);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

317. Bipartite Graph (DFS)

Understanding the Problem NEW

Check whether a graph is 'bipartite' — meaning its nodes can be split into two groups such that every edge connects a node in one group to a node in the other (no edge connects two nodes within the same group). Equivalent to asking: can the graph be properly colored with just 2 colors?

Brute Force Approach NEW

Try every possible way of splitting nodes into two groups and check if all edges cross between groups — 2^n possible splits, exponential.

Why We Optimize NEW

Use DFS (or BFS), **assigning alternating colors** as you traverse: color the starting node, then every neighbour must get the *opposite* color; if you ever find a neighbour that's already colored and it **matches** the current node's color (rather than being the opposite), the graph isn't bipartite — an edge would then connect two same-colored nodes, violating the definition. This checks bipartiteness in a single O(V+E) traversal, instead of an exponential search over every possible 2-way split, by building the coloring incrementally and detecting a contradiction the moment one arises.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if graph is bipartite (2-colorable).

Visual Diagram

```
Bipartite:
       0 --- 1
       |     |
       3 --- 2

       Color assignment:
       0: red, 1: blue, 2: red, 3: blue

       Non-bipartite (odd cycle):
       0 --- 1
```

```
|     |
3 --- 2
Cannot color with 2 colors
```

Solution Approach

Concept: DFS with alternating colors.

Algorithm

```
Algorithm IS_BIPARTITE(graph):
           color = new int[V] // -1: uncolored, 0: red, 1: blue

            For i = 0 to V-1:
                If color[i] == -1:
                    If !dfs(i, 0, color, graph):
                        Return false
            Return true

       Algorithm dfs(node, c, color, graph):
           color[node] = c
           For neighbor in graph[node]:
               If color[neighbor] == -1:
                   If !dfs(neighbor, 1-c, color, graph):
                       Return false
               Else If color[neighbor] == c:
                   Return false
           Return true
```

Java Implementation

```
class Solution {
           public boolean isBipartite(int[][] graph) {
               int n = graph.length;
               int[] color = new int[n];
               Arrays.fill(color, -1);

                 for (int i = 0; i < n; i++) {
                     if (color[i] == -1) {
                         if (!dfs(i, 0, color, graph)) {
                              return false;
                         }
                     }
                 }
                 return true;

         }

          private boolean dfs(int node, int c, int[] color, int[][] graph) {
              color[node] = c;
```

```
            for (int neighbor : graph[node]) {
                if (color[neighbor] == -1) {
                    if (!dfs(neighbor, 1 - c, color, graph)) {
                         return false;
                    }
                } else if (color[neighbor] == c) {
                    return false;
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

318. Cycle Detection in Directed Graph (DFS)

Understanding the Problem NEW

Detect a cycle in a *directed* graph — trickier than the undirected version (309-310), since in a directed graph, encountering an already-visited node doesn't automatically mean a cycle (it might just be a different, valid path converging on a node you've already finished exploring elsewhere).

Brute Force Approach NEW

Reusing the undirected 'track only the immediate parent' technique directly would give wrong answers here, since directed graphs can have multiple valid paths converging on the same node without that being a cycle — so a naive reuse of 309-310's technique doesn't work.

Why We Optimize NEW

Use **three-state coloring** during DFS instead of a simple visited/unvisited flag: 'white' (not yet visited), 'gray' (currently being explored, i.e. on the current DFS path / call stack), and 'black' (fully finished exploring). A cycle exists specifically if you ever encounter a **'gray' node** — meaning you've looped back to something still actively on your current path (a true cycle), as opposed to a 'black' node (already fully explored via some other path, which is fine and expected in directed graphs with converging paths). This distinguishes genuine back-edges (cycles) from harmless cross-edges/forward-edges that are completely normal in directed graphs, something the simpler undirected-graph technique can't tell apart.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect cycles in directed graph.

Visual Diagram

```
Graph with cycle:
       0 → 1 → 2 → 0

       DFS with 3 states:
       0: unvisited
```

```
       1: visiting (in current recursion stack)
       2: visited (processed)

       0: visiting
       1: visiting
       2: visiting
       0: visiting → cycle found

       Graph without cycle:
       0 → 1 → 2
       All nodes processed → no cycle
```

Solution Approach

Concept: DFS with 3-state coloring.

Algorithm

```
Algorithm HAS_CYCLE_DIRECTED(graph):
           state = new int[V] // 0: unvisited, 1: visiting, 2: visited

           For i = 0 to V-1:
               If state[i] == 0:
                   If dfs(i, state, graph):
                        Return true
           Return false

       Algorithm dfs(node, state, graph):
           state[node] = 1 // visiting
           For neighbor in graph[node]:
               If state[neighbor] == 0:
                   If dfs(neighbor, state, graph): Return true
               Else If state[neighbor] == 1:
                   Return true // back edge
           state[node] = 2 // processed
           Return false
```

Java Implementation

```
class Solution {
           public boolean hasCycle(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
               }

                 int[] state = new int[n]; // 0: unvisited, 1: visiting, 2: visited
```

```
            for (int i = 0; i < n; i++) {
                if (state[i] == 0) {
                    if (dfs(i, state, graph)) {
                        return true;
                    }
                }
            }
            return false;
        }

        private boolean dfs(int node, int[] state, List<List<Integer>> graph) {
            state[node] = 1; // visiting

            for (int neighbor : graph.get(node)) {
                if (state[neighbor] == 0) {
                    if (dfs(neighbor, state, graph)) {
                        return true;
                    }
                } else if (state[neighbor] == 1) {
                    return true; // back edge
                }
            }
            state[node] = 2; // processed
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

319. Topological Sort

Understanding the Problem NEW

Given a Directed Acyclic Graph (DAG — a directed graph with no cycles), find a 'topological order' — an ordering of all nodes such that every edge points from an earlier node to a later one in the ordering (like a valid sequence of course prerequisites).

Brute Force Approach NEW

Try every possible permutation of nodes and check if it satisfies every edge's ordering constraint — n! permutations, completely infeasible.

Why We Optimize NEW

Run DFS from every unvisited node, and the moment a node **finishes** being fully explored (all its descendants have been visited), push it onto a stack; once all nodes have been processed this way, popping the entire stack gives a valid topological order. This works because a node only gets pushed after everything reachable from it has already been pushed (and will therefore come out of the stack pop *after* — meaning positioned earlier in the resulting order — everything it points to), guaranteeing the ordering constraint for every edge. This achieves the topological order in a single O(V+E) DFS pass (using the 'black/finished' three-state idea from problem 318 to know when a node is truly done), instead of the factorial brute-force permutation check.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Order vertices so all edges go forward.

Visual Diagram

```
Graph:
       0 → 1 → 3
       ↓   ↓
       2   4

       Topological Order: [0, 1, 2, 3, 4] or [0, 2, 1, 3, 4]
```

```
       DFS Algorithm:
       Visit 0 → visit 1 → visit 3 → visit 4 → visit 2
       Add to stack: 4, 3, 2, 1, 0
       Reverse: 0, 1, 2, 3, 4
```

Solution Approach

Concept: DFS with stack.

Algorithm

```
Algorithm TOPO_SORT_DFS(graph):
           visited = boolean[V]
           stack = Stack()

           For i = 0 to V-1:
               If !visited[i]:
                   dfs(i, visited, stack, graph)

           Return stack (reversed)

       Algorithm dfs(node, visited, stack, graph):
           visited[node] = true
           For neighbor in graph[node]:
               If !visited[neighbor]:
                   dfs(neighbor, visited, stack, graph)
           stack.push(node)
```

Java Implementation

```
class Solution {
           public List<Integer> topologicalSort(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
               }

                 boolean[] visited = new boolean[n];
                 Stack<Integer> stack = new Stack<>();

                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {

                      dfs(i, graph, visited, stack);
                  }
              }
```

```
            List<Integer> result = new ArrayList<>();
            while (!stack.isEmpty()) {
                result.add(stack.pop());
            }
            return result;
        }

        private void dfs(int node, List<List<Integer>> graph, boolean[] visited,
Stack<Integer> sta
            visited[node] = true;
            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs(neighbor, graph, visited, stack);
                }
            }
            stack.push(node);
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

320. Kahn's Algorithm

Understanding the Problem NEW

An alternative way to compute topological order (same goal as problem 319), this time using BFS instead of DFS.

Brute Force Approach NEW

Same factorial permutation-checking alternative as 319 would apply here too if attempted naively.

Why We Optimize NEW

Kahn's Algorithm: first compute the **in-degree** (number of incoming edges) for every node; any node with in-degree 0 has no unmet prerequisites, so it can safely go first — enqueue all such nodes initially. Then repeatedly dequeue a node, add it to the topological order, and 'remove' its outgoing edges by decrementing the in-degree of each neighbour; any neighbour whose in-degree drops to 0 as a result gets enqueued next. This BFS-based approach achieves the same O(V+E) result as DFS-based topological sort (319), but has the added benefit of naturally detecting cycles too (problem 321) — if not all nodes get processed by the end, some nodes' in-degrees never dropped to 0, meaning a cycle prevented them from ever becoming 'ready.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Topological sort using indegree.

Visual Diagram

```
Graph:
       0 → 1 → 3
       ↓   ↓
       2   4

       Indegrees:
       0:0, 1:1, 2:1, 3:1, 4:1

       Queue: [0]
       Pop 0, add neighbors: indegree[1]=0, indegree[2]=0
```

```
       Queue: [1,2]
       Pop 1, add neighbors: indegree[3]=0, indegree[4]=0
       Queue: [2,3,4]
       Pop 2,3,4

       Order: [0, 1, 2, 3, 4]
```

Solution Approach

Concept: Queue with indegree 0 nodes.

Algorithm

```
Algorithm KAHN(graph):
           indegree = new int[V]
           For each edge u→v:
               indegree[v]++

           queue = Queue()
           For i = 0 to V-1:
               If indegree[i] == 0:
                   queue.add(i)

           result = []
           While queue not empty:
               node = queue.poll()
               result.add(node)
               For neighbor in graph[node]:
                   indegree[neighbor]--
                   If indegree[neighbor] == 0:
                       queue.add(neighbor)

           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> topologicalSortKahn(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[n];
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);

                  indegree[edge[1]]++;
              }

              Queue<Integer> queue = new LinkedList<>();
```

```
            for (int i = 0; i < n; i++) {
                if (indegree[i] == 0) {
                    queue.add(i);
                }
            }

            List<Integer> result = new ArrayList<>();
            while (!queue.isEmpty()) {
                int node = queue.poll();
                result.add(node);
                for (int neighbor : graph.get(node)) {
                    indegree[neighbor]--;
                    if (indegree[neighbor] == 0) {
                        queue.add(neighbor);
                    }
                }
            }
            return result.size() == n ? result : new ArrayList<>();
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

321. Cycle Detection in Directed Graph (Kahn's)

Understanding the Problem NEW

Detect whether a directed graph has a cycle, using Kahn's Algorithm's BFS-based approach rather than the DFS three-color approach from problem 318.

Brute Force Approach NEW

Same exponential-path-enumeration alternative as any cycle-detection problem.

Why We Optimize NEW

Run Kahn's Algorithm (320) exactly as normal, but simply **count how many nodes get successfully processed** (added to the order). If that count equals the total number of nodes, the graph is acyclic (every node eventually had its dependencies satisfied); if it's *less* than the total, some nodes never had their in-degree reach 0 — meaning they're stuck in a cycle where each depends on another that never gets 'freed.' This gives an alternative cycle-detection method to problem 318's DFS coloring, useful particularly because it naturally falls out of running Kahn's Algorithm anyway if you needed the topological order too.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect cycle using Kahn's algorithm.

Visual Diagram

```
Graph with cycle:
       0 → 1 → 2 → 0

       Indegrees:
       0:1, 1:1, 2:1
       No node with indegree 0 → cycle exists

       Graph without cycle:
       0 → 1 → 2

       Indegrees:
       0:0, 1:1, 2:1
       Queue: [0]
```

```
       Pop 0 → indegree[1]=0 → queue: [1]
       Pop 1 → indegree[2]=0 → queue: [2]
       Pop 2 → all processed → no cycle
```

Solution Approach

Concept: Kahn's algorithm with count.

Algorithm

```
Algorithm HAS_CYCLE_KAHN(graph):
           indegree = new int[V]
           For each edge u→v: indegree[v]++

           queue = Queue with indegree 0 nodes
           count = 0

           While queue not empty:
               node = queue.poll()
               count++
               For neighbor in graph[node]:
                   indegree[neighbor]--
                   If indegree[neighbor] == 0:
                       queue.add(neighbor)

           Return count != V // cycle exists
```

Java Implementation

```
class Solution {
           public boolean hasCycleKahn(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[n];
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
                   indegree[edge[1]]++;
               }

                 Queue<Integer> queue = new LinkedList<>();

              for (int i = 0; i < n; i++) {
                  if (indegree[i] == 0) {
                      queue.add(i);
                  }
              }

              int count = 0;
```

```
            while (!queue.isEmpty()) {
                int node = queue.poll();
                count++;
                for (int neighbor : graph.get(node)) {
                    indegree[neighbor]--;
                    if (indegree[neighbor] == 0) {
                         queue.add(neighbor);
                    }
                }
            }
            return count != n;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

322. Course Schedule - I

Understanding the Problem NEW

Given a number of courses and a list of prerequisite pairs, determine if it's possible to finish all courses (i.e., is there a valid completion order, or do the prerequisites form an impossible cycle?).

Brute Force Approach NEW

Try every possible course order and check if all prerequisites are satisfied — n! orderings, infeasible.

Why We Optimize NEW

This is directly Cycle Detection in a Directed Graph (318 or 321) in disguise: model courses as nodes and prerequisites as directed edges, then the courses can all be finished **if and only if the graph has no cycle** (a cycle would mean course A needs B, which needs C, which circles back to needing A — an impossible deadlock). Reusing either the DFS three-color method or Kahn's Algorithm answers this in O(V+E), rather than exploring every possible course ordering — recognizing the real-world framing as a direct instance of an already-solved graph problem is the entire insight here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if all courses can be completed.

Visual Diagram

```
numCourses = 2, prerequisites = [[1,0]]
       Graph: 0 → 1
       Can complete: Yes

       numCourses = 2, prerequisites = [[1,0],[0,1]]
       Graph: 0 → 1 → 0 (cycle)
       Can complete: No

       Check for cycle in directed graph
```

Solution Approach

Concept: Cycle detection in directed graph.

Algorithm

```
Algorithm CAN_FINISH(numCourses, prerequisites):
           Build graph
           Return !hasCycle_Kahn(graph)
```

Java Implementation

```
class Solution {
           public boolean canFinish(int numCourses, int[][] prerequisites) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[numCourses];
               for (int i = 0; i < numCourses; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] pre : prerequisites) {
                   graph.get(pre[1]).add(pre[0]);
                   indegree[pre[0]]++;
               }

                 Queue<Integer> queue = new LinkedList<>();
                 for (int i = 0; i < numCourses; i++) {
                     if (indegree[i] == 0) {
                         queue.add(i);
                     }
                 }

                 int count = 0;
                 while (!queue.isEmpty()) {
                     int course = queue.poll();
                     count++;
                     for (int next : graph.get(course)) {
                         indegree[next]--;
                         if (indegree[next] == 0) {
                              queue.add(next);
                         }
                     }
                 }
                 return count == numCourses;
           }
       }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V+E)

323. Course Schedule - II

Understanding the Problem NEW

Same setup as Course Schedule-I, but now return an actual **valid order** to complete all courses (or an empty result if impossible).

Brute Force Approach NEW

Same exponential permutation search as a naive approach to 322.

Why We Optimize NEW

This is directly Topological Sort (319 or 320) in disguise — the valid course order *is* a topological order of the prerequisite graph, and if Kahn's Algorithm (320) doesn't manage to process every node, that's simultaneously your signal that it's impossible (same cycle-detection fact from problem 321). One algorithm run answers both 'is it possible' and 'what's a valid order,' achieved in O(V+E), reusing topological sort rather than solving 'finishability' and 'find an order' as two separate problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Return order of courses to complete all.

Visual Diagram

```
numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
       Graph: 0 → 1 → 3
              0 → 2 → 3

       Topological Order: [0, 1, 2, 3] or [0, 2, 1, 3]

       Kahn's Algorithm:
       Indegrees: 0:0, 1:1, 2:1, 3:2
       Queue: [0]
       Pop 0 → indegree[1]=0, indegree[2]=0
       Queue: [1,2]
       Pop 1 → indegree[3]=1
       Pop 2 → indegree[3]=0
       Queue: [3]
```

```
       Pop 3

       Order: [0, 1, 2, 3]
```

Solution Approach

Concept: Kahn's algorithm for topological order.

Algorithm

```
Algorithm FIND_ORDER(numCourses, prerequisites):
           Build graph and indegree
           Kahn's algorithm
           Return order if count == numCourses else []
```

Java Implementation

```
class Solution {
           public int[] findOrder(int numCourses, int[][] prerequisites) {
               List<List<Integer>> graph = new ArrayList<>();
               int[] indegree = new int[numCourses];
               for (int i = 0; i < numCourses; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] pre : prerequisites) {
                   graph.get(pre[1]).add(pre[0]);
                   indegree[pre[0]]++;
               }

                 Queue<Integer> queue = new LinkedList<>();
                 for (int i = 0; i < numCourses; i++) {
                     if (indegree[i] == 0) {
                         queue.add(i);
                     }
                 }

                 int[] order = new int[numCourses];
                 int index = 0;
                 while (!queue.isEmpty()) {
                     int course = queue.poll();
                     order[index++] = course;

                  for (int next : graph.get(course)) {
                      indegree[next]--;
                      if (indegree[next] == 0) {
                          queue.add(next);
                      }
                  }
              }
              return index == numCourses ? order : new int[0];
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V+E)

324. Find Eventual Safe States

Understanding the Problem NEW

Given a directed graph, find all 'safe' nodes — nodes from which every possible path eventually leads to a terminal node (a node with no outgoing edges), meaning you can never get stuck in an infinite cycle no matter which path you take from a safe node.

Brute Force Approach NEW

For every node, explore every possible path starting from it and check if all of them terminate — exponential, since the number of paths from a node can grow very large.

Why We Optimize NEW

A node is unsafe **if and only if it's part of a cycle, or can reach a cycle** — so this is a cycle-detection problem in disguise, solvable with the same **three-state DFS coloring** from problem 318: a node is 'safe' if DFS from it never encounters a 'gray' (currently-on-path) node — meaning no cycle is reachable from it. Reusing the three-color technique (with memoization of each node's safe/unsafe status to avoid recomputing for shared sub-paths) determines every node's safety in a single O(V+E) pass, instead of exponentially exploring every path from every node.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find nodes that eventually reach terminal nodes.

Visual Diagram

```
Graph:
       0 → 1 → 2
       ↓   ↓
       3   4

       Terminal nodes: 3, 4
       Safe nodes: 0,1,2,3,4
```

```
       DFS with 3 states:
       0: unvisited
       1: visiting (potentially unsafe)
       2: processed (safe)
```

Solution Approach

Concept: DFS with 3-state coloring.

Algorithm

```
Algorithm SAFE_STATES(graph):
           state = new int[V] // 0: unvisited, 1: visiting, 2: safe
           result = []

           For i = 0 to V-1:
               If dfs(i, state, graph):
                   result.add(i)
           Return result

       Algorithm dfs(node, state, graph):
           If state[node] == 1: Return false
           If state[node] == 2: Return true

           state[node] = 1 // visiting
           For neighbor in graph[node]:
               If !dfs(neighbor, state, graph):
                   Return false
           state[node] = 2 // safe
           Return true
```

Java Implementation

```
class Solution {
           public List<Integer> eventualSafeNodes(int[][] graph) {
               int n = graph.length;
               int[] state = new int[n]; // 0: unvisited, 1: visiting, 2: safe
               List<Integer> result = new ArrayList<>();

                 for (int i = 0; i < n; i++) {
                     if (dfs(i, state, graph)) {
                         result.add(i);
                     }
                 }
                 return result;
           }
           private boolean dfs(int node, int[] state, int[][] graph) {

              if (state[node] == 1) return false;
              if (state[node] == 2) return true;
```

```
            state[node] = 1; // visiting
            for (int neighbor : graph[node]) {
                if (!dfs(neighbor, state, graph)) {
                    return false;
                }
            }
            state[node] = 2; // safe
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

325. Alien Dictionary

Understanding the Problem NEW

Given a list of words from an alien language sorted according to that language's (unknown) alphabetical order, determine what that order actually is.

Brute Force Approach NEW

Try every possible ordering of the alphabet's letters and check which one is consistent with the given sorted word list — with up to 26 letters, that's 26! possible orderings, astronomically infeasible.

Why We Optimize NEW

Compare each pair of **adjacent words** in the list: the first position where they differ tells you that the earlier word's letter must come **before** the later word's letter in the alien alphabet — build a directed graph of these letter-precedence constraints from every adjacent word pair, then the actual alien alphabet order is simply a **topological sort** (319/320) of this constraint graph. This turns an exponential guess-the-full-ordering problem into 'extract pairwise ordering constraints, then topologically sort them' — directly reusing the topological sort technique, with an added subtlety: if a later word is a *strict prefix* of an earlier word (e.g. 'abc' appearing after 'ab'), that's an immediate contradiction (a prefix must always come first), which needs an explicit special check since the letter-by-letter comparison wouldn't otherwise catch it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find order of letters in alien language.

Visual Diagram

```
Words: ["wrt", "wrf", "er", "ett", "rftt"]
       Output: "wertf"

       Compare adjacent words:
       wrt vs wrf → t < f
       wrf vs er → w < e
       er vs ett → r < t
```

```
       ett vs rftt → e < r

       Graph: t→f, w→e, r→t, e→r
       Topological: w→e→r→t→f = "wertf"
```

Solution Approach

Concept: Build graph and topological sort.

Algorithm

```
Algorithm ALIEN_DICTIONARY(words):
           graph = new HashMap<Character, Set<Character>>()
           indegree = new HashMap<Character, Integer>()

            // Initialize all characters
            For each word, add all chars to graph

            // Compare adjacent words
            For i = 0 to words.length-2:
                word1 = words[i], word2 = words[i+1]
                For j = 0 to min(word1.length, word2.length):
                    If word1[j] != word2[j]:
                        If word2[j] not in graph[word1[j]]:
                            graph[word1[j]].add(word2[j])
                            indegree[word2[j]]++
                        Break

            // Topological sort
            Queue = all chars with indegree 0
            Build result
            Return result if all chars included else ""
```

Java Implementation

```
class Solution {
           public String alienOrder(String[] words) {
               Map<Character, Set<Character>> graph = new HashMap<>();
               Map<Character, Integer> indegree = new HashMap<>();

                 // Initialize
                 for (String word : words) {
                     for (char c : word.toCharArray()) {
                         graph.putIfAbsent(c, new HashSet<>());
                         indegree.putIfAbsent(c, 0);
                     }
                 }
                 // Build graph

             for (int i = 0; i < words.length - 1; i++) {
```

```
                String word1 = words[i], word2 = words[i+1];
                int minLen = Math.min(word1.length(), word2.length());
                boolean found = false;

                for (int j = 0; j < minLen; j++) {
                    char c1 = word1.charAt(j), c2 = word2.charAt(j);
                    if (c1 != c2) {
                        if (!graph.get(c1).contains(c2)) {
                            graph.get(c1).add(c2);
                            indegree.put(c2, indegree.get(c2) + 1);
                        }
                        found = true;
                        break;
                    }
                }
                if (!found && word1.length() > word2.length()) {
                    return "";
                }
            }

            // Topological sort
            Queue<Character> queue = new LinkedList<>();
            for (char c : indegree.keySet()) {
                if (indegree.get(c) == 0) {
                    queue.add(c);
                }
            }

            StringBuilder result = new StringBuilder();
            while (!queue.isEmpty()) {
                char c = queue.poll();
                result.append(c);
                for (char next : graph.get(c)) {
                    indegree.put(next, indegree.get(next) - 1);
                    if (indegree.get(next) == 0) {
                        queue.add(next);
                    }
                }
            }
            return result.length() == indegree.size() ? result.toString() : "";
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * L) | Space Complexity: O(V+E)

326. Shortest Path in Undirected Graph with Unit Weight

Understanding the Problem NEW

Find the shortest distance from a source node to every other node, in an undirected graph where every edge has the same weight (unit weight, i.e. an unweighted graph).

Brute Force Approach NEW

You could run a separate search from the source to each individual target node — O(V) separate traversals, each O(V+E), giving O(V×(V+E)) overall, hugely redundant since a single traversal can actually discover distances to everyone at once.

Why We Optimize NEW

Plain **BFS from the source** already solves this optimally in one pass: since BFS explores level by level, and every edge costs the same 'one step,' the level at which BFS reaches a node *is* exactly that node's shortest distance from the source. A single O(V+E) BFS traversal gives every node's shortest distance simultaneously — the 'unit weight' condition is precisely what makes plain BFS sufficient here; the moment edges have different weights, this stops working and you need Dijkstra's Algorithm (328) instead, which is exactly the next problem's motivation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest distance from source to all nodes.

Visual Diagram

```
Graph:
       0 --- 1
       |     |
       3 --- 2

       BFS from 0:
       dist[0] = 0
```

```
       dist[1] = 1, dist[3] = 1
       dist[2] = 2

       BFS queue:
       [0] → pop 0, add 1,3
       [1,3] → pop 1, add 2
       [3,2] → pop 3
       [2] → pop 2
```

Solution Approach

Concept: BFS for unweighted graph.

Algorithm

```
Algorithm SHORTEST_PATH_UNWEIGHTED(graph, source):
           dist = new int[V]
           Arrays.fill(dist, -1)
           queue = Queue()
           dist[source] = 0
           queue.add(source)

           While queue not empty:
               node = queue.poll()
               For neighbor in graph[node]:
                   If dist[neighbor] == -1:
                       dist[neighbor] = dist[node] + 1
                       queue.add(neighbor)
           Return dist
```

Java Implementation

```
class Solution {
           public int[] shortestPath(int n, int[][] edges, int source) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(edge[1]);
                   graph.get(edge[1]).add(edge[0]);
               }

                  int[] dist = new int[n];
                  Arrays.fill(dist, -1);

              Queue<Integer> queue = new LinkedList<>();
              dist[source] = 0;
              queue.add(source);
```

```
            while (!queue.isEmpty()) {
                int node = queue.poll();
                for (int neighbor : graph.get(node)) {
                    if (dist[neighbor] == -1) {
                         dist[neighbor] = dist[node] + 1;
                         queue.add(neighbor);
                    }
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

327. Shortest Path in DAG

Understanding the Problem NEW

Find the shortest path from a source to every other node, in a **Directed Acyclic Graph (DAG)** where edges can have different (possibly non-unit) weights.

Brute Force Approach NEW

You could try Dijkstra's Algorithm (328) directly, which would work — but it does more work than necessary here, since a DAG's acyclic structure allows for something even simpler and faster.

Why We Optimize NEW

Because the graph is acyclic, you can first compute a **topological order** (319) of the nodes, then process nodes *in that order*, 'relaxing' each outgoing edge (i.e., updating a neighbour's best-known distance if going through the current node gives a shorter path) — since topological order guarantees you always finish computing a node's final shortest distance before you ever need to use it to update something later in the order, one single pass through the topologically sorted nodes suffices. This achieves O(V+E) — genuinely faster than Dijkstra's O(E log V) — by exploiting the DAG's acyclic property (no need for the priority-queue-based 'always process the currently-closest unprocessed node next' strategy that general graphs with cycles require).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest path in Directed Acyclic Graph.

Visual Diagram

```
DAG:
       0 → 1 (weight 5)
       0 → 2 (weight 3)
       1 → 3 (weight 6)
       2 → 3 (weight 2)
       2 → 4 (weight 4)
       3 → 4 (weight 1)
```

```
Topological order: [0, 2, 1, 3, 4]
Distances from 0:
dist[0] = 0
dist[1] = 5
dist[2] = 3
dist[3] = min(5+6=11, 3+2=5) = 5
dist[4] = min(3+4=7, 5+1=6) = 6
```

Solution Approach

Concept: Topological order + relaxation.

Algorithm

```
Algorithm SHORTEST_PATH_DAG(graph, source):
           topo = topologicalSort(graph)
           dist = new int[V]
           Arrays.fill(dist, INF)
           dist[source] = 0

           For node in topo:
               If dist[node] != INF:
                   For neighbor, weight in graph[node]:
                       dist[neighbor] = min(dist[neighbor], dist[node] + weight)

           Return dist
```

Java Implementation

```
class Solution {
           public int[] shortestPathDAG(int n, int[][] edges, int source) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(new int[]{edge[1], edge[2]});
               }

                 // Topological sort
                 boolean[] visited = new boolean[n];
                 Stack<Integer> stack = new Stack<>();
                 for (int i = 0; i < n; i++) {
                     if (!visited[i]) {
                         dfs(i, graph, visited, stack);
                     }
                 }

              // Shortest path
              int[] dist = new int[n];
```

```
            Arrays.fill(dist, Integer.MAX_VALUE);
            dist[source] = 0;

            while (!stack.isEmpty()) {
                int node = stack.pop();
                if (dist[node] != Integer.MAX_VALUE) {
                    for (int[] neighbor : graph.get(node)) {
                        int next = neighbor[0], weight = neighbor[1];
                        dist[next] = Math.min(dist[next], dist[node] + weight);
                    }
                }
            }
            return dist;
        }

        private void dfs(int node, List<List<int[]>> graph, boolean[] visited,
Stack<Integer> stack
            visited[node] = true;
            for (int[] neighbor : graph.get(node)) {
                if (!visited[neighbor[0]]) {
                    dfs(neighbor[0], graph, visited, stack);
                }
            }
            stack.push(node);
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

328. Dijkstra's Algorithm

Understanding the Problem NEW

Find the shortest path from a source to every other node in a graph with **non-negative** edge weights (edges can now have different costs, and the graph can have cycles, unlike problem 327's DAG restriction).

Brute Force Approach NEW

Try every possible path from source to each target and compare total weights — exponential, since the number of distinct paths in a graph with cycles can be unbounded (well, bounded but combinatorially huge) unless a cycle is avoided during enumeration, which adds further complexity to a naive approach.

Why We Optimize NEW

Dijkstra's Algorithm: repeatedly pick the **currently closest unprocessed node** (using a Min-Heap/Priority Queue to always retrieve it in O(log V)), and 'relax' all of its outgoing edges (update neighbours' distances if a shorter path is found through this node) — since you always process the globally-closest remaining node next, once a node is processed, its shortest distance is guaranteed final (any other path to it would have to go through an already-processed, farther node, which can't yield something shorter given non-negative weights). This greedy 'always finalize the closest remaining node next' strategy, powered by a heap, achieves O((V+E) log V) instead of exponential path enumeration — but critically depends on **non-negative weights** for its greedy correctness guarantee, which is exactly why negative edges need Bellman-Ford (336) instead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Shortest path in weighted graph (non-negative edges).

Visual Diagram

```
Graph:
       0 → 1 (4)
       0 → 2 (1)
       1 → 3 (1)
```

```
       2 → 1 (2)
       2 → 3 (5)
       3 → 4 (3)

       From 0:
       dist[0] = 0
       dist[2] = 1
       dist[1] = min(4, 1+2=3) = 3
       dist[3] = min(3+1=4, 1+5=6) = 4
       dist[4] = 7

       Priority Queue:
       (0,0) → pop 0
       (1,2) → pop 2
       (3,1) → pop 1
       (4,3) → pop 3
       (7,4) → pop 4
```

Solution Approach

Concept: Priority Queue with distance relaxation.

Algorithm

```
Algorithm DIJKSTRA(graph, source):
           dist = new int[V]
           Arrays.fill(dist, INF)
           dist[source] = 0
           pq = PriorityQueue((d, node) -> d)
           pq.add((0, source))

           While pq not empty:
               (d, node) = pq.poll()
               If d > dist[node]: continue
               For neighbor, weight in graph[node]:
                   If dist[node] + weight < dist[neighbor]:
                       dist[neighbor] = dist[node] + weight
                       pq.add((dist[neighbor], neighbor))

           Return dist
```

Java Implementation

```
class Solution {
           public int[] dijkstra(int n, int[][] edges, int source) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
```

```
                graph.get(edge[0]).add(new int[]{edge[1], edge[2]});

                graph.get(edge[1]).add(new int[]{edge[0], edge[2]});
            }

            int[] dist = new int[n];
            Arrays.fill(dist, Integer.MAX_VALUE);
            dist[source] = 0;

            PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
            pq.add(new int[]{0, source});

            while (!pq.isEmpty()) {
                int[] curr = pq.poll();
                int d = curr[0], node = curr[1];
                if (d > dist[node]) continue;

                for (int[] neighbor : graph.get(node)) {
                    int next = neighbor[0], weight = neighbor[1];
                    if (dist[node] + weight < dist[next]) {
                        dist[next] = dist[node] + weight;
                        pq.add(new int[]{dist[next], next});
                    }
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V)

329. Why Priority Queue is Used in Dijkstra

Understanding the Problem NEW

This isn't a new problem — it's a dedicated explanation of *why* Dijkstra's Algorithm specifically needs a Priority Queue (Min-Heap) rather than, say, a plain queue or a simple array scan.

Brute Force Approach NEW

You could scan the entire list of unprocessed nodes each time to find the current minimum-distance one — O(V) per selection, O(V²) total across all selections, which is actually a valid (if slower) variant of Dijkstra's Algorithm sometimes used for very dense graphs.

Why We Optimize NEW

A Priority Queue retrieves the current minimum in O(log V) instead of O(V), which matters because Dijkstra performs this 'find the closest unprocessed node' operation up to V times — using a heap brings the total down to O((V+E) log V) instead of O(V²), a meaningful improvement for sparse graphs (where E is much smaller than V²). This connects directly back to the Heaps chapter's core lesson (problem 211): a heap is exactly the right structure whenever you repeatedly need 'the current best/smallest candidate' from a changing set, which is precisely Dijkstra's repeated need throughout its execution.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understand the role of priority queue in Dijkstra.

Visual Diagram

```
Without Priority Queue (using simple queue):
       0 → 1 (10)
       0 → 2 (5)
       2 → 1 (2)

       Queue approach:
       Process 0 → dist[1]=10, dist[2]=5
       Process 2 → dist[1]=7 (update)
```

```
Process 1 → no updates
Works but process order matters

With Priority Queue:
Always process smallest distance first
Guarantees optimal distance when popped
No need to process nodes multiple times

Example:
pq: (0,0)
Pop 0 → add (5,2), (10,1)
Pop 2 → add (7,1)
Pop 1 → final
```

Java Implementation

```
class Solution {
           public int[] dijkstraWithPQ(int n, int[][] edges, int source) {
               // Same as Dijkstra implementation above
               // PQ ensures O((V+E)logV) complexity
               // Without PQ, it would be O(V²)

                // PQ is used because:
                // 1. Always extracts minimum distance
                // 2. Each node processed once
                // 3. Efficient O(log V) extraction
                // 4. Handles dynamic updates
                // 5. Guarantees correctness
                return new int[0];
            }
       }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V)

330. Shortest Path in a Binary Maze

Understanding the Problem NEW

Given a binary grid (0s are open cells, 1s are blocked), find the shortest path from top-left to bottom-right, moving through adjacent open cells (including diagonals, in some variants).

Brute Force Approach NEW

Try every possible path via exhaustive DFS exploration, tracking the shortest one found — exponential, since the number of paths through a grid can grow very large.

Why We Optimize NEW

Since every move costs the same '1 step' (unweighted, like problem 326), plain **BFS from the start cell** finds the shortest path directly — the level at which BFS reaches the destination cell is exactly the shortest path length. This is Shortest Path with Unit Weight (326), applied to an implicit grid-graph (306's 'grid is a graph in disguise' idea) instead of an explicit adjacency list — no new algorithm needed, just recognizing the grid as an unweighted graph where BFS is the right tool.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest path in a binary maze.

Visual Diagram

```
Grid:
       0 0 0
       1 1 0
       0 0 0
       Start: (0,0), End: (2,2)

       Path: (0,0)→(0,1)→(0,2)→(1,2)→(2,2) = 5 steps

       BFS with 4 directions:
       Queue: [(0,0)]
       Level 1: (0,1)
```

```
       Level 2: (0,2)
       Level 3: (1,2)
       Level 4: (2,2)
```

Solution Approach

Concept: BFS on grid.

Algorithm

```
Algorithm SHORTEST_PATH_MAZE(grid, start, end):
           m = grid.length, n = grid[0].length
           visited = boolean[m][n]
           queue = Queue()
           queue.add(start)
           visited[start[0]][start[1]] = true
           dist = 0

           While queue not empty:
               size = queue.size()
               For i = 0 to size-1:
                   cell = queue.poll()
                   If cell == end: Return dist
                   For each direction:
                       If valid and grid[x][y] == 0 and !visited:
                           queue.add((x,y))
                           visited[x][y] = true
               dist++
           Return -1
```

Java Implementation

```
class Solution {
           public int shortestPath(int[][] grid, int[] start, int[] end) {
               int m = grid.length, n = grid[0].length;
               boolean[][] visited = new boolean[m][n];
               Queue<int[]> queue = new LinkedList<>();
               queue.add(start);
               visited[start[0]][start[1]] = true;
               int dist = 0;
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 while (!queue.isEmpty()) {
                     int size = queue.size();

                  for (int i = 0; i < size; i++) {
                      int[] cell = queue.poll();
                      if (cell[0] == end[0] && cell[1] == end[1]) {
                          return dist;
                      }
```

```
                    for (int[] dir : dirs) {
                        int x = cell[0] + dir[0];
                        int y = cell[1] + dir[1];
                        if (x >= 0 && x < m && y >= 0 && y < n &&
                            grid[x][y] == 0 && !visited[x][y]) {
                            queue.add(new int[]{x, y});
                            visited[x][y] = true;
                        }
                    }
                }
                dist++;
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

331. Path with Minimum Effort

Understanding the Problem NEW

Given a grid where moving between adjacent cells costs the absolute difference in their values ('effort'), find a path from top-left to bottom-right that minimizes the **maximum single-step effort** along the path (not the total effort — just the worst single step).

Brute Force Approach NEW

Try every possible path and compute each one's maximum step-effort, tracking the overall minimum — exponential path exploration.

Why We Optimize NEW

This needs a **modified Dijkstra's Algorithm**: instead of tracking and minimizing the *sum* of edge weights along a path (as ordinary Dijkstra does), track and minimize the **maximum** edge weight seen so far along a path — when 'relaxing' an edge, a path is considered better if it reduces the *worst step so far*, not the total. The same priority-queue-driven 'always expand the currently best path next' structure from Dijkstra applies unchanged, just with the comparison/update rule changed from 'sum' to 'max' — a good example of how Dijkstra's greedy priority-queue framework generalizes to different path-cost definitions, not just simple additive weights.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find path with minimum maximum effort.

Visual Diagram

```
Grid:
       [1,2,2]
       [3,8,2]
       [5,3,5]
       Effort = max absolute difference

       Path: (0,0)→(0,1)→(0,2)→(1,2)→(2,2)
       Efforts: |2-1|=1, |2-2|=0, |8-2|=6, |5-8|=3
```

```
       Max effort = 6

       Dijkstra variation:
       dist[i][j] = min max effort to reach (i,j)
       Priority queue by effort
```

Solution Approach

Concept: Modified Dijkstra.

Algorithm

```
Algorithm MINIMUM_EFFORT(heights):
           m = heights.length, n = heights[0].length
           dist = new int[m][n]
           Arrays.fill(dist, INF)
           dist[0][0] = 0
           pq = PriorityQueue((effort, x, y))
           pq.add((0, 0, 0))

           While pq not empty:
               (effort, x, y) = pq.poll()
               If effort > dist[x][y]: continue
               If x == m-1 && y == n-1: return effort
               For each direction:
                   newEffort = max(effort, abs(heights[x][y] - heights[nx][ny]))
                   If newEffort < dist[nx][ny]:
                       dist[nx][ny] = newEffort
                       pq.add((newEffort, nx, ny))
           Return -1
```

Java Implementation

```
class Solution {
           public int minimumEffortPath(int[][] heights) {
               int m = heights.length, n = heights[0].length;
               int[][] dist = new int[m][n];
               for (int[] row : dist) {
                   Arrays.fill(row, Integer.MAX_VALUE);
               }
               dist[0][0] = 0;

                 PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] -
b[0]);
                 pq.add(new int[]{0, 0, 0});
                 int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 while (!pq.isEmpty()) {

                  int[] curr = pq.poll();
```

```
                int effort = curr[0], x = curr[1], y = curr[2];
                if (effort > dist[x][y]) continue;
                if (x == m-1 && y == n-1) return effort;

                for (int[] dir : dirs) {
                    int nx = x + dir[0];
                    int ny = y + dir[1];
                    if (nx >= 0 && nx < m && ny >= 0 && ny < n) {
                        int newEffort = Math.max(effort,
                            Math.abs(heights[x][y] - heights[nx][ny]));
                        if (newEffort < dist[nx][ny]) {
                            dist[nx][ny] = newEffort;
                            pq.add(new int[]{newEffort, nx, ny});
                        }
                    }
                }
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n log(m*n)) | Space Complexity: O(m*n)

332. Cheapest Flights Within K Stops

Understanding the Problem NEW

Find the cheapest flight route from a source city to a destination, allowed at most K stopover cities along the way (a constraint that ordinary shortest-path algorithms don't account for).

Brute Force Approach NEW

Try every possible route with up to K+1 flights and compare total costs — the number of possible routes can grow exponentially with the number of cities and allowed stops.

Why We Optimize NEW

Dijkstra's Algorithm doesn't directly work here because its greedy 'always finalize the closest node' guarantee assumes you don't care *how many edges* a path used — but here, a route's validity depends on stop count, not just cost. Instead, use a **Bellman-Ford-style approach**: relax all edges, but only allow up to **K+1 rounds** of relaxation (each round representing one additional flight taken), tracking the best cost reachable within that many flights specifically. This bounded-rounds relaxation (rather than Dijkstra's unbounded greedy processing) is exactly what correctly respects the 'at most K stops' constraint — a case where the general shortest-path problem needs a different algorithm once an extra constraint (hop count) is added on top of just minimizing total cost.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find cheapest flight with at most k stops.

Visual Diagram

```
n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], k=1
       Path: 0→1→2 cost=200

       BFS with DP:
       dp[node][stops] = min cost
       Bellman-Ford variant
```

```
       dp[0][0] = 0
       For stops = 1 to k+1:
           For each flight:
               dp[flight.to][stops] = min(dp[flight.to][stops],
                                          dp[flight.from][stops-1] + cost)
```

Solution Approach

Concept: Bellman-Ford with stop constraint.

Algorithm

```
Algorithm CHEAPEST_FLIGHT(n, flights, src, dst, k):
           dist = new int[n]
           Arrays.fill(dist, INF)
           dist[src] = 0

           For i = 1 to k+1:
               temp = dist.clone()
               For each flight (from, to, price):
                   If dist[from] != INF:
                       temp[to] = min(temp[to], dist[from] + price)
               dist = temp
               If dist[dst] updated: track

           Return dist[dst] == INF ? -1 : dist[dst]
```

Java Implementation

```
class Solution {
           public int findCheapestPrice(int n, int[][] flights, int src, int dst,
int k) {
               int[] dist = new int[n];
               Arrays.fill(dist, Integer.MAX_VALUE);
               dist[src] = 0;

                 for (int i = 0; i <= k; i++) {
                     int[] temp = Arrays.copyOf(dist, n);
                     for (int[] flight : flights) {
                         int from = flight[0], to = flight[1], price = flight[2];
                         if (dist[from] != Integer.MAX_VALUE) {
                             temp[to] = Math.min(temp[to], dist[from] + price);
                         }
                     }
                     dist = temp;
                 }
                 return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
           }
       }
```

Time & Space Complexity

Time Complexity: O(k * E) | Space Complexity: O(V)

333. Network Delay Time

Understanding the Problem NEW

Given a network of nodes and directed weighted edges representing signal transmission times, find the time for a signal sent from a source node to reach *every* node in the network (or determine it's impossible).

Brute Force Approach NEW

Try every possible path from the source to every target node and take the minimum for each — exponential path enumeration.

Why We Optimize NEW

This is directly Dijkstra's Algorithm (328) applied as-is: run Dijkstra from the source node to get the shortest distance to every other node, and the answer is simply the **maximum** of all these shortest distances (since the whole network 'finishes receiving the signal' only once the slowest-to-reach node gets it) — or -1/ impossible if any node's distance remains unreachable. No new technique needed here — just directly applying Dijkstra and taking the max of the resulting distance array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Time for signal to reach all nodes.

Visual Diagram

```
Graph:
       1 → 2 (1)
       2 → 3 (1)
       1 → 3 (2)

       From 1:
       dist[1]=0, dist[2]=1, dist[3]=2
       Max distance = 2

       Dijkstra from source
       Return max distance
```

Solution Approach

Concept: Dijkstra's algorithm.

Algorithm

```
Algorithm NETWORK_DELAY(times, n, k):
           graph = adjacency list
           dist = Dijkstra(k)
           maxTime = max(dist[1..n])
           Return maxTime == INF ? -1 : maxTime
```

Java Implementation

```
class Solution {
           public int networkDelayTime(int[][] times, int n, int k) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i <= n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] time : times) {
                   graph.get(time[0]).add(new int[]{time[1], time[2]});
               }

                 int[] dist = new int[n + 1];
                 Arrays.fill(dist, Integer.MAX_VALUE);
                 dist[k] = 0;

                 PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] -
b[1]);
                 pq.add(new int[]{k, 0});

                 while (!pq.isEmpty()) {
                     int[] curr = pq.poll();
                     int node = curr[0], d = curr[1];
                     if (d > dist[node]) continue;

                    for (int[] neighbor : graph.get(node)) {
                        int next = neighbor[0], weight = neighbor[1];
                        if (dist[node] + weight < dist[next]) {
                            dist[next] = dist[node] + weight;
                            pq.add(new int[]{next, dist[next]});
                        }
                    }

             }

             int maxTime = 0;
             for (int i = 1; i <= n; i++) {
                 if (dist[i] == Integer.MAX_VALUE) return -1;
                 maxTime = Math.max(maxTime, dist[i]);
```

```
            }
            return maxTime;
        }
    }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V+E)

334. Number of Ways to Arrive at Destination

Understanding the Problem NEW

Find not just the shortest path length from source to destination, but **how many distinct shortest paths** exist (paths that all tie for the minimum possible distance).

Brute Force Approach NEW

Enumerate every possible path, group them by total distance, and count how many share the minimum — exponential.

Why We Optimize NEW

Extend Dijkstra's Algorithm with an extra **'ways' counter** per node: whenever you relax an edge and find a *strictly shorter* path to a neighbour, reset that neighbour's ways-count to match the current node's ways-count (a completely new best path type); but if you find a path that *ties* the neighbour's current best distance exactly, **add** the current node's ways-count to the neighbour's existing ways-count (an additional equally-good path has been found). This piggybacks path-counting onto the same Dijkstra traversal, achieving the same O((V+E) log V) complexity while tracking one extra piece of bookkeeping per node, rather than needing a separate exponential enumeration of all paths.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of shortest paths to destination.

Visual Diagram

```
Graph:
       0 → 1 (1)
       0 → 2 (2)
       1 → 2 (1)
       1 → 3 (3)
       2 → 3 (1)

       Number of shortest paths from 0 to 3:
       0→1→2→3 (1+1+1=3)
       0→2→3 (2+1=3)
```

```
       0→1→3 (1+3=4)
       0→1→2→3 (1+1+1=3)
       Total: 2 paths

       Modified Dijkstra:
       dist[v] = shortest distance
       ways[v] = number of ways
       If dist[node] + weight < dist[next]:
           dist[next] = dist[node] + weight
           ways[next] = ways[node]
       If dist[node] + weight == dist[next]:
           ways[next] += ways[node]
```

Solution Approach

Concept: Dijkstra with ways tracking.

Algorithm

```
Algorithm COUNT_WAYS(n, roads):
           Build graph
           dist = new long[n], ways = new long[n]
           dist[source] = 0, ways[source] = 1
           pq = PriorityQueue()

           While pq not empty:
               (d, node) = pq.poll()
               If d > dist[node]: continue
               For (neighbor, weight) in graph[node]:
                   If dist[node] + weight < dist[neighbor]:
                       dist[neighbor] = dist[node] + weight
                       ways[neighbor] = ways[node]
                       pq.add((dist[neighbor], neighbor))
                   Else If dist[node] + weight == dist[neighbor]:
                       ways[neighbor] = (ways[neighbor] + ways[node]) % MOD
           Return ways[dst]
```

Java Implementation

```
class Solution {
           public int countPaths(int n, int[][] roads) {
               final int MOD = 1_000_000_007;
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());

              }
              for (int[] road : roads) {
                  graph.get(road[0]).add(new int[]{road[1], road[2]});
                  graph.get(road[1]).add(new int[]{road[0], road[2]});
```

```
            }

            long[] dist = new long[n];
            long[] ways = new long[n];
            Arrays.fill(dist, Long.MAX_VALUE);
            dist[0] = 0;
            ways[0] = 1;

            PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) ->
                Long.compare(a[1], b[1]));
            pq.add(new long[]{0, 0});

            while (!pq.isEmpty()) {
                long[] curr = pq.poll();
                int node = (int) curr[0];
                long d = curr[1];
                if (d > dist[node]) continue;

                for (int[] neighbor : graph.get(node)) {
                    int next = neighbor[0], weight = neighbor[1];
                    if (dist[node] + weight < dist[next]) {
                        dist[next] = dist[node] + weight;
                        ways[next] = ways[node];
                        pq.add(new long[]{next, dist[next]});
                    } else if (dist[node] + weight == dist[next]) {
                        ways[next] = (ways[next] + ways[node]) % MOD;
                    }
                }
            }
            return (int) ways[n-1];
        }
    }
```

Time & Space Complexity

Time Complexity: O((V+E) log V) | Space Complexity: O(V+E)

335. Minimum Steps to Reach End from Start

Understanding the Problem NEW

Given a starting value and target value, and a set of allowed transformation operations, find the minimum number of operations to reach the target from the start — a generalization of Word Ladder (314) to arbitrary 'states' and 'operations' rather than specifically words and letter changes.

Brute Force Approach NEW

Try every possible sequence of operations exhaustively — exponential, since the number of possible operation sequences grows very quickly.

Why We Optimize NEW

Exactly the same insight as Word Ladder: model each reachable 'state' as a graph node, with an edge to every state reachable via one allowed operation, and the minimum number of operations is exactly the **shortest path** length from the start state to the target state in this implicit graph — found via plain BFS (since every operation, like every word-transformation step, costs one unit), generating each state's neighbours on the fly via the allowed operations rather than building the whole state graph upfront. This is the general template Word Ladder was a specific instance of: 'minimum operations to transform X into Y' problems are almost always BFS-on-an-implicit-state-graph problems in disguise.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum steps to reach target using operations.

Visual Diagram

```
Start: 0, Target: 10
       Operations: 2x, x+1, x-1

       BFS:
       Level 0: [0]
       Level 1: [1, -1, 0] → [1, -1]
       Level 2: [2, 0, 2, -2] → [2, -2]
       Level 3: [4, 1, 4, -4] → [4, -4]
```

```
Level 4: [8, 3, 8, -8] → [8, -8]
Level 5: [16, 7, 16, -16] → [16,7,16,-16]
Continue until 10 found
```

Solution Approach

Concept: BFS on state space.

Algorithm

```
Algorithm MIN_STEPS(start, target, operations):
           visited = Set()
           queue = Queue()
           queue.add(start)
           visited.add(start)
           steps = 0

           While queue not empty:
               size = queue.size()
               For i = 0 to size-1:
                   node = queue.poll()
                   If node == target: Return steps
                   For each operation:
                       next = operation(node)
                       If !visited.contains(next):
                           visited.add(next)
                           queue.add(next)
               steps++
           Return -1
```

Java Implementation

```
class Solution {
           public int minSteps(int start, int target, int[] ops) {
               Set<Integer> visited = new HashSet<>();
               Queue<Integer> queue = new LinkedList<>();
               queue.add(start);
               visited.add(start);
               int steps = 0;

                 while (!queue.isEmpty()) {
                     int size = queue.size();
                     for (int i = 0; i < size; i++) {
                         int node = queue.poll();
                         if (node == target) return steps;

                         for (int op : ops) {

                           int next = applyOp(node, op);
                           if (next >= 0 && next <= 1000 && !
```

```
visited.contains(next)) {
                                visited.add(next);
                                queue.add(next);
                            }
                        }
                    }
                    steps++;
                }
                return -1;
            }

            private int applyOp(int node, int op) {
                // Example operations: 0: +1, 1: -1, 2: *2
                if (op == 0) return node + 1;
                if (op == 1) return node - 1;
                if (op == 2) return node * 2;
                return node;
            }
        }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

336. Bellman Ford Algorithm

Understanding the Problem NEW

Find shortest paths from a source node in a graph that may contain **negative** edge weights (something Dijkstra's Algorithm cannot correctly handle, since its greedy 'finalize the closest node' guarantee breaks down when a later negative edge could still improve an already-finalized node's distance).

Brute Force Approach NEW

You might try running Dijkstra anyway — but it can produce incorrect results with negative edges, since its core greedy assumption (processing nodes in increasing distance order guarantees correctness) is specifically violated by negative weights.

Why We Optimize NEW

Bellman-Ford takes a different approach: relax *every* edge in the graph, repeated for **V-1 rounds** (where V is the number of nodes) — this many rounds is guaranteed sufficient because the longest possible shortest path (without cycles) can have at most V-1 edges, so after V-1 full relaxation rounds, every shortest distance is guaranteed to have stabilized, even accounting for negative edges. As a valuable bonus, running one *additional* (V-th) relaxation round and checking if **any** distance still improves reveals whether the graph contains a **negative cycle** (a cycle whose total weight is negative, which would make 'shortest path' undefined, since you could loop around it forever to keep decreasing the total). This costs O(V×E) — slower than Dijkstra's O((V+E) log V) — but correctly handles negative weights, which Dijkstra's greedy priority-queue approach fundamentally cannot.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Shortest path with negative edges (no negative cycle).

Visual Diagram

```
Graph:
       0 → 1 (4)
```

```
0 → 2 (1)
2 → 1 (2)
1 → 3 (1)
2 → 3 (5)

From 0:
dist[0] = 0
Iteration 1: dist[2]=1, dist[1]=4
Iteration 2: dist[1]=3, dist[3]=4
Iteration 3: dist[3]=4 (no change)

Negative cycle detection:
If any relaxation in V-th iteration → negative cycle
```

Solution Approach

Concept: V-1 relaxations.

Algorithm

```
Algorithm BELLMAN_FORD(graph, source):
           dist = new int[V]
           Arrays.fill(dist, INF)
           dist[source] = 0

           For i = 1 to V-1:
               For each edge (u, v, weight):
                   If dist[u] != INF and dist[u] + weight < dist[v]:
                       dist[v] = dist[u] + weight

           // Check negative cycle
           For each edge (u, v, weight):
               If dist[u] != INF and dist[u] + weight < dist[v]:
                   Return false // Negative cycle exists

           Return true
```

Java Implementation

```
class Solution {
           public int[] bellmanFord(int n, int[][] edges, int source) {
               int[] dist = new int[n];
               Arrays.fill(dist, Integer.MAX_VALUE);
               dist[source] = 0;

                 // V-1 iterations
                 for (int i = 1; i < n; i++) {
                     for (int[] edge : edges) {
                         int u = edge[0], v = edge[1], w = edge[2];
                         if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v])
```

```
{
                            dist[v] = dist[u] + w;
                        }
                    }

            }

            // Check negative cycle
            for (int[] edge : edges) {
                int u = edge[0], v = edge[1], w = edge[2];
                if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                    return null; // Negative cycle exists
                }
            }
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: O(V*E) | Space Complexity: O(V)

337. Floyd Warshall Algorithm

Understanding the Problem NEW

Find the shortest path between *every* pair of nodes in a graph (all-pairs shortest paths), rather than just from a single source.

Brute Force Approach NEW

Run Dijkstra's Algorithm (or Bellman-Ford, if negative edges are present) separately from every single node as a source — O(V) separate shortest-path computations, each O((V+E) log V) or O(V×E), giving O(V² log V) or O(V²×E) overall.

Why We Optimize NEW

Floyd-Warshall takes a fundamentally different, dynamic-programming-based approach: for every pair (i, j), check whether routing *through* an intermediate node k gives a shorter path than the current best known — trying every possible intermediate node k, one at a time, updating the entire distance matrix after considering each k. This achieves all-pairs shortest paths in O(V³) with a wonderfully simple triple-nested loop, and — unlike running Dijkstra V separate times — it naturally handles negative edge weights too (though still not negative cycles), making it a genuinely different algorithmic strategy (incremental 'allow one more intermediate node at a time' DP) rather than just 'repeat single-source Dijkstra V times.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

All-pairs shortest path.

Visual Diagram

```
Graph:
       0 → 1 (4)
       0 → 2 (1)
       2 → 1 (2)
       1 → 3 (1)
       2 → 3 (5)
```

```
       Matrix:
       dist[0][1] = min(4, 1+2=3) = 3
       dist[0][3] = min(5, 1+2+1=4) = 4
       dist[2][3] = min(5, 2+1=3) = 3

       For k from 0 to V-1:
           For i from 0 to V-1:
               For j from 0 to V-1:
                   dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

Solution Approach

Concept: Dynamic programming with intermediate nodes.

Algorithm

```
Algorithm FLOYD_WARSHALL(graph):
           dist = graph matrix
           For k = 0 to V-1:
               For i = 0 to V-1:
                   For j = 0 to V-1:
                       If dist[i][k] + dist[k][j] < dist[i][j]:
                           dist[i][j] = dist[i][k] + dist[k][j]
           Return dist
```

Java Implementation

```
class Solution {
           public int[][] floydWarshall(int[][] graph) {
               int n = graph.length;
               int[][] dist = new int[n][n];
               for (int i = 0; i < n; i++) {
                   for (int j = 0; j < n; j++) {
                        dist[i][j] = graph[i][j];
                        if (i != j && dist[i][j] == 0) {
                            dist[i][j] = Integer.MAX_VALUE / 2;
                        }
                   }
               }

                 for (int k = 0; k < n; k++) {
                     for (int i = 0; i < n; i++) {
                         for (int j = 0; j < n; j++) {
                             if (dist[i][k] + dist[k][j] < dist[i][j]) {
                                 dist[i][j] = dist[i][k] + dist[k][j];
                             }
                         }
                     }

              }
```

```
            return dist;
        }
    }
```

Time & Space Complexity

Time Complexity: $O(V^3)$ | Space Complexity: $O(V^2)$

338. Find City with Smallest Number of Neighbors

Understanding the Problem NEW

Given a graph of cities with weighted edges and a distance threshold, find the city that has the **fewest** other cities reachable within that threshold distance (with ties broken by choosing the city with the largest index).

Brute Force Approach NEW

Run a separate shortest-path computation (Dijkstra) from every city and count reachable cities within the threshold for each — O(V) Dijkstra runs, each O((V+E) log V).

Why We Optimize NEW

Since you need shortest distances between *every* pair of cities anyway (to count each city's reachable neighbours), this is a natural fit for **Floyd-Warshall** (337) computed once for the whole graph — after that single $O(V^3)$ computation, counting each city's reachable-within-threshold neighbours is just a simple O(V) scan per city ($O(V^2)$ total) over the completed distance matrix. This reuses Floyd-Warshall's all-pairs result directly, rather than running V separate single-source searches, since the problem's need for 'every city's reachability count' aligns exactly with what an all-pairs shortest-path computation already provides.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find city with minimum reachable cities within threshold.

Visual Diagram

```
n=4, edges=[[0,1,3],[1,2,1],[1,3,4],[2,3,1]], threshold=4
       Floyd Warshall distances:
       From 0: [0,3,4,5] → 2 cities (1,2)
       From 1: [3,0,1,2] → 3 cities (0,2,3)
       From 2: [4,1,0,1] → 3 cities (0,1,3)
       From 3: [5,2,1,0] → 2 cities (1,2)
       Result: 0 (tie break by larger index? Actually smaller index with min count)
```

Solution Approach

Concept: Floyd Warshall + counting.

Algorithm

```
Algorithm FIND_CITY(n, edges, threshold):
           dist = FloydWarshall(edges)
           minCount = INF, city = -1

           For i = 0 to n-1:
               count = 0
               For j = 0 to n-1:
                   If i != j and dist[i][j] <= threshold:
                       count++
               If count <= minCount:
                   minCount = count
                   city = i
           Return city
```

Java Implementation

```
class Solution {
           public int findTheCity(int n, int[][] edges, int threshold) {
               int[][] dist = new int[n][n];
               for (int i = 0; i < n; i++) {
                   Arrays.fill(dist[i], Integer.MAX_VALUE / 2);
                   dist[i][i] = 0;
               }
               for (int[] edge : edges) {
                   dist[edge[0]][edge[1]] = edge[2];
                   dist[edge[1]][edge[0]] = edge[2];
               }

                 // Floyd Warshall
                 for (int k = 0; k < n; k++) {
                     for (int i = 0; i < n; i++) {
                         for (int j = 0; j < n; j++) {
                             dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k]
[j]);
                         }
                     }
                 }

                 int minCount = n;
                 int result = -1;
                 for (int i = 0; i < n; i++) {
                     int count = 0;

                   for (int j = 0; j < n; j++) {
                       if (i != j && dist[i][j] <= threshold) {
```

```
                        count++;
                    }
                }
                if (count <= minCount) {
                    minCount = count;
                    result = i;
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

339-340. Minimum Spanning Tree & Prim's Algorithm

Understanding the Problem NEW

Given a connected, undirected, weighted graph, find a **Minimum Spanning Tree (MST)** — a subset of edges that connects all nodes together (forming a tree, no cycles) with the minimum possible total edge weight.

Brute Force Approach NEW

Try every possible subset of edges that forms a spanning tree and compare total weights — the number of possible spanning trees can be astronomically large (exponential) for even moderately sized graphs.

Why We Optimize NEW

Prim's Algorithm builds the MST greedily, growing a single connected tree one edge at a time: starting from any node, repeatedly add the **cheapest edge** that connects the current tree to a new, not-yet-included node (using a Min-Heap/ Priority Queue — the same heap-driven 'always grab the current best candidate' pattern from Dijkstra — to efficiently find this cheapest connecting edge at each step). Because adding the cheapest available connecting edge at every step never creates a worse result than any alternative choice (a classic greedy-exchange argument), this builds a guaranteed-optimal MST in O(E log V), instead of exploring the exponentially many possible spanning trees directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find MST with minimum total edge weight.

Visual Diagram

```
Graph:
       0 --4-- 1
       |       |
       3       2
       2 --3-- 3 --1-- 4

       Prim's Algorithm from 0:
```

```
       MST: 0--3 (2), 3--4 (1), 4--2 (3), 2--1 (2)
       Total: 2+1+3+2 = 8
```

Solution Approach

Concept: Priority queue with visited nodes.

Algorithm

```
Algorithm PRIM(graph):
           visited = boolean[V]
           pq = PriorityQueue((weight, node))
           pq.add((0, 0))
           totalWeight = 0

           While pq not empty:
               (w, node) = pq.poll()
               If visited[node]: continue
               visited[node] = true
               totalWeight += w
               For neighbor, weight in graph[node]:
                   If !visited[neighbor]:
                       pq.add((weight, neighbor))
           Return totalWeight
```

Java Implementation

```
class Solution {
           public int primMST(int n, int[][] edges) {
               List<List<int[]>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
               }
               for (int[] edge : edges) {
                   graph.get(edge[0]).add(new int[]{edge[1], edge[2]});
                   graph.get(edge[1]).add(new int[]{edge[0], edge[2]});
               }

                 boolean[] visited = new boolean[n];
                 PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] -
b[1]);
                 pq.add(new int[]{0, 0});
                 int totalWeight = 0;

                 while (!pq.isEmpty()) {
                     int[] curr = pq.poll();

                  int node = curr[0], weight = curr[1];
                  if (visited[node]) continue;
```

```
                visited[node] = true;
                totalWeight += weight;

                for (int[] neighbor : graph.get(node)) {
                    if (!visited[neighbor[0]]) {
                        pq.add(new int[]{neighbor[0], neighbor[1]});
                    }
                }
            }
            return totalWeight;
        }
    }
```

Time & Space Complexity

Time Complexity: O(E log V) | Space Complexity: O(V+E)

341-342. Disjoint Set (Union Find)

Understanding the Problem NEW

A **Disjoint Set (Union-Find)** is a specialized data structure for efficiently answering 'are these two elements in the same group?' and 'merge these two groups together' — critical for the next several MST and connectivity-focused problems.

Brute Force Approach NEW

Without a specialized structure, you could represent groups as explicit lists and merge them by concatenating lists — checking group membership might be O(n) per query (scanning a list), and merging two groups could cost O(size of the smaller list) each time, adding up to potentially $O(n^2)$ across many merge operations.

Why We Optimize NEW

Union-Find represents each group as a **tree** where every node points to a 'parent,' with the group's identity determined by its tree's root — **'find'** follows parent pointers up to the root (answering 'which group am I in'), and **'union'** merges two groups by attaching one tree's root under the other's. Two crucial optimizations make this extremely fast: **path compression** (during 'find,' make every visited node point directly to the root, flattening future lookups) and **union by rank/size** (always attach the smaller/shallower tree under the larger/deeper one's root, keeping trees flat). Together, these bring both operations down to nearly O(1) amortized (technically $O(\alpha(n))$, the inverse Ackermann function — effectively constant for any realistic input size) — a dramatic improvement over the naive list-based approach's O(n) per operation, and this structure becomes essential for Kruskal's Algorithm (343) and many later connectivity problems in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Union-Find data structure for connectivity.

Visual Diagram

```
Elements: 0,1,2,3,4
       Initial: Each is own parent

       Union(0,1): parent[0]=1
       Union(2,3): parent[2]=3
       Union(1,3): parent[1]=3
       Find(0): 0→1→3 → returns 3
       Find(2): 2→3 → returns 3
       Connected: 0 and 2 are connected

       Union by Rank:
       - Smaller tree attached to larger
       - O(α(n)) amortized
```

Solution Approach

Concept: Parent array with path compression.

Algorithm

```
Algorithm DSU:
           parent = new int[n]
           rank = new int[n]

           find(x):
               If parent[x] != x:
                    parent[x] = find(parent[x])
               Return parent[x]

           union(x, y):
               px = find(x), py = find(y)
               If px == py: return
               If rank[px] < rank[py]: parent[px] = py
               Else If rank[px] > rank[py]: parent[py] = px
               Else: parent[py] = px, rank[px]++
```

Java Implementation

```
class DSU {
           private int[] parent;
           private int[] rank;

           DSU(int n) {
               parent = new int[n];
               rank = new int[n];
               for (int i = 0; i < n; i++) {
                   parent[i] = i;
               }
```

```
        }

        public int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]); // Path compression
            }
            return parent[x];

        }

        public void union(int x, int y) {
            int px = find(x);
            int py = find(y);
            if (px == py) return;

            if (rank[px] < rank[py]) {
                parent[px] = py;
            } else if (rank[px] > rank[py]) {
                parent[py] = px;
            } else {
                parent[py] = px;
                rank[px]++;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(α(n)) amortized | Space Complexity: O(n)

343. Kruskal's Algorithm

Understanding the Problem NEW

An alternative way to find a Minimum Spanning Tree (same goal as Prim's Algorithm, 339-340), this time by considering edges globally sorted by weight rather than growing one connected tree.

Brute Force Approach NEW

Same exponential spanning-tree enumeration as a naive approach to Prim's problem.

Why We Optimize NEW

Kruskal's Algorithm: sort *all* edges in the graph by weight, ascending, then process them one at a time in that order, adding an edge to the MST only if its two endpoints are **currently in different components** (checked instantly using Union-Find's 'find' operation from problems 341-342) — skipping any edge that would connect two nodes already in the same component (which would create a cycle, not allowed in a tree). This greedy 'take the globally cheapest edge that doesn't create a cycle' strategy, powered by Union-Find for near-instant cycle checking, achieves O(E log E) (dominated by the sort), and is often preferred over Prim's when the edge list is more naturally available than an adjacency structure, or when the graph is sparse.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

MST using Kruskal's algorithm.

Visual Diagram

```
Graph:
       0 --4-- 1
       |       |
       3       2
       2 --3-- 3 --1-- 4

       Sort edges by weight:
       (3,4) = 1
```

```
       (0,3) = 2
       (2,3) = 3
       (2,4) = 3
       (0,1) = 4
       (1,2) = 2

       Union:
       (3,4) → union 3,4
       (0,3) → union 0,3
       (2,3) → union 2,3 (connects 2 to 3)
       (2,4) → already connected
       (0,1) → union 0,1 (connects 1 to 0)
       Total: 1+2+3+2 = 8
```

Solution Approach

Concept: Sort edges, union if different components.

Algorithm

```
Algorithm KRUSKAL(edges, n):
           Sort edges by weight
           dsu = DSU(n)
           totalWeight = 0
           count = 0

           For each edge (u, v, w):
               If dsu.find(u) != dsu.find(v):
                   dsu.union(u, v)
                   totalWeight += w
                   count++
                   If count == n-1: break
           Return totalWeight
```

Java Implementation

```
class Solution {
           public int kruskalMST(int n, int[][] edges) {
               Arrays.sort(edges, (a, b) -> a[2] - b[2]);
               DSU dsu = new DSU(n);
               int totalWeight = 0;
               int count = 0;

                 for (int[] edge : edges) {
                     int u = edge[0], v = edge[1], w = edge[2];
                     if (dsu.find(u) != dsu.find(v)) {
                         dsu.union(u, v);

                      totalWeight += w;
                      count++;
```

```
                    if (count == n - 1) break;
                }
            }
            return totalWeight;
        }
    }
```

Time & Space Complexity

Time Complexity: O(E log E) | Space Complexity: O(V)

344. Number of Operations to Make Network Connected

Understanding the Problem NEW

Given a network of computers with some direct connections, and the ability to move an existing cable to connect two currently-disconnected computers, find the minimum number of cable moves needed to connect the entire network (or -1 if there aren't enough cables).

Brute Force Approach NEW

Try every possible way of removing and re-adding cables and check which sequence connects everything with the fewest moves — combinatorially expensive without structure.

Why We Optimize NEW

Use **Union-Find**: process every existing connection, unioning the two computers it connects — after processing all connections, count how many **separate components** remain (using Union-Find's structure to identify this) and how many 'extra' cables exist beyond what's needed to connect within each component (a component with k computers only needs k-1 cables to stay connected; any additional cables within it are 'spare' and can be relocated). If there are at least (number of components - 1) spare cables available, the answer is simply (number of components - 1) — since connecting C separate components together always takes exactly C-1 new connecting cables, mirroring how a spanning tree connects components with the minimum number of edges. This reduces a seemingly complex cable-rearrangement problem to counting components via Union-Find, an O(E) process.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum connections to connect all computers.

Visual Diagram

```
n=4, connections=[[0,1],[0,2],[1,2]]
       Output: 1 (connect 3 to any)
```

```
Count components:
Component 1: [0,1,2]
Component 2: [3]
Need: components-1 = 1 extra connection

If extra cables < components-1: impossible
Extra cables = edges - (n - components)
```

Solution Approach

Concept: DSU to count components.

Algorithm

```
Algorithm MAKE_CONNECTED(n, connections):
           If connections.length < n-1: Return -1
           dsu = DSU(n)
           For each connection: dsu.union(u, v)

           components = 0
           For i = 0 to n-1:
               If dsu.find(i) == i: components++

           Return components - 1
```

Java Implementation

```
class Solution {
           public int makeConnected(int n, int[][] connections) {
               if (connections.length < n - 1) return -1;

                 DSU dsu = new DSU(n);
                 for (int[] conn : connections) {
                     dsu.union(conn[0], conn[1]);
                 }

                 int components = 0;
                 for (int i = 0; i < n; i++) {
                     if (dsu.find(i) == i) {
                         components++;
                     }
                 }
                 return components - 1;
           }
      }
```

Time & Space Complexity

Time Complexity: O(E α(n)) | Space Complexity: O(n)

345. Most Stones Removed with Same Row/Column

Understanding the Problem NEW

Given stones on a 2D grid, you can remove a stone if it shares a row or column with another remaining stone; find the maximum number of stones that can be removed.

Brute Force Approach NEW

Try every possible order and choice of stone removals and track the maximum removable — combinatorially expensive, since removal order and choices interact in complex ways.

Why We Optimize NEW

Reframe using **Union-Find**, but cleverly: instead of treating stones as the elements to union, union **rows and columns** together whenever a stone connects them (using a shared numbering scheme, e.g. row indices and negated/offset column indices to keep them distinct in the same Union-Find structure) — any group of stones connected (directly or transitively) via shared rows/columns forms one Union-Find component, and within any single component, you can always remove all but one stone (by removing them in the right order, always leaving one 'anchor' stone until the very end). So the answer is simply `(total stones) - (number of distinct components)`, computed via one pass unioning rows/columns per stone — turning a seemingly complex order-dependent removal puzzle into a straightforward component-counting problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum stones that can be removed.

Visual Diagram

```
Stones: [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]
       Groups (connected by row/column):
       Group 1: [0,0], [0,1], [1,0], [2,0]? etc.

       DSU on rows and columns:
       For each stone (i,j):
```

```
            union(i, j+n) // row and column nodes

        Components = count of unique parents
        Removed = stones.length - components
```

Solution Approach

Concept: DSU with row and column nodes.

Algorithm

```
Algorithm REMOVE_STONES(stones):
            n = stones.length
            dsu = DSU(2 * maxCoord)

            For each stone (x, y):
                dsu.union(x, y + maxCoord)

            components = 0
            For each stone (x, y):
                If dsu.find(x) == x: components++
            Return n - components
```

Java Implementation

```
class Solution {
            public int removeStones(int[][] stones) {
                int maxCoord = 10000;
                DSU dsu = new DSU(2 * maxCoord + 1);

                  for (int[] stone : stones) {
                      dsu.union(stone[0], stone[1] + maxCoord);
                  }

                  Set<Integer> components = new HashSet<>();
                  for (int[] stone : stones) {
                      components.add(dsu.find(stone[0]));
                  }
                  return stones.length - components.size();
            }
        }
```

Time & Space Complexity

Time Complexity: O(n α(n)) | Space Complexity: O(n)

346. Accounts Merge

Understanding the Problem NEW

Given a list of accounts, each with a name and a list of associated emails, merge accounts that share at least one email (since they must belong to the same actual person), combining all their emails into one sorted list per merged account.

Brute Force Approach NEW

Compare every pair of accounts to check for a shared email, merging matches, and repeat until no more merges are possible — O(accounts²) comparisons, potentially requiring multiple passes since merging two accounts could newly connect them to a third.

Why We Optimize NEW

Use **Union-Find over emails**: treat every distinct email as an element, and for each account, union all of its emails together (since they all belong to the same person by virtue of appearing in the same account) — after processing every account this way, any two emails ending up in the same Union-Find component belong to the same actual person, regardless of how many accounts originally separated them. Finally, group emails by their root/component and attach the appropriate name to each group. This turns a potentially multi-pass, pairwise-comparison merging problem into a single pass of Union-Find operations (near-O(1) each), followed by one grouping pass — directly reusing Union-Find's efficient 'these things are connected, merge them' capability for a very different-looking real-world problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge accounts with common emails.

Visual Diagram

```
Accounts:
      ["John", "johnsmith@mail.com", "john_newyork@mail.com"]
      ["John", "johnsmith@mail.com", "john00@mail.com"]
      ["Mary", "mary@mail.com"]
```

```
      ["John", "johnnybravo@mail.com"]

      Output:
      ["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"]
      ["John", "johnnybravo@mail.com"]
      ["Mary", "mary@mail.com"]

      DSU on emails
      Map email → name
```

Solution Approach

Concept: DSU on emails.

Algorithm

```
Algorithm ACCOUNTS_MERGE(accounts):
          parent = HashMap<String, String>
          emailToName = HashMap<String, String>

          For each account:
              For each email in account[1:]:
                  emailToName.put(email, account[0])
                  parent.put(email, email)
                  If it's not first email:
                      union(firstEmail, email)

          groups = HashMap<String, List<String>>
          For each email in parent:
              root = find(email)
              groups.putIfAbsent(root, new ArrayList<>())
              groups.get(root).add(email)

          result = []
          For each group:
              sorted emails, add name
          Return result
```

Java Implementation

```
class Solution {
          public List<List<String>> accountsMerge(List<List<String>> accounts) {
              Map<String, String> parent = new HashMap<>();
              Map<String, String> emailToName = new HashMap<>();

                 // Initialize DSU
                 for (List<String> account : accounts) {
                     String name = account.get(0);
                     for (int i = 1; i < account.size(); i++) {
                         String email = account.get(i);
```

```
                    parent.put(email, email);

                emailToName.put(email, name);
            }
        }

        // Union emails in same account
        for (List<String> account : accounts) {
            String firstEmail = account.get(1);
            for (int i = 2; i < account.size(); i++) {
                union(parent, firstEmail, account.get(i));
            }
        }

        // Group emails
        Map<String, List<String>> groups = new HashMap<>();
        for (String email : parent.keySet()) {
            String root = find(parent, email);
            groups.computeIfAbsent(root, k -> new ArrayList<>()).add(email);
        }

        // Build result
        List<List<String>> result = new ArrayList<>();
        for (List<String> emails : groups.values()) {
            Collections.sort(emails);
            List<String> account = new ArrayList<>();
            account.add(emailToName.get(emails.get(0)));
            account.addAll(emails);
            result.add(account);
        }
        return result;
    }

    private String find(Map<String, String> parent, String x) {
        if (!parent.get(x).equals(x)) {
            parent.put(x, find(parent, parent.get(x)));
        }
        return parent.get(x);
    }

    private void union(Map<String, String> parent, String x, String y) {
        String px = find(parent, x);
        String py = find(parent, y);
        if (!px.equals(py)) {
            parent.put(px, py);
        }
    }
}
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

347. Number of Islands II

Understanding the Problem NEW

Given a grid that starts entirely water, and a sequence of 'add land' operations (one cell at a time), report the number of distinct islands **after each individual operation** — requiring an efficient way to track connectivity as land is added incrementally, rather than recomputing from scratch each time.

Brute Force Approach NEW

After each single land addition, re-run a full connected-components scan (like problem 306) over the entire grid — O(rows×cols) per addition, giving O(operations × rows × cols) overall, extremely wasteful since only one cell changes between consecutive queries.

Why We Optimize NEW

Use **Union-Find**, incrementally: when a new land cell is added, initially count it as a brand new island (increment a running island count), then check its up/down/left/right neighbours — for every neighbour that's *already land* and in a *different* Union-Find component, union them together and **decrement** the running island count by 1 (since two separate islands just merged into one). This maintains the running island count incrementally, giving each query an O(α(n)) (near-constant) answer instead of a full O(rows×cols) rescan — exploiting the fact that only local changes near the newly-added cell can possibly affect connectivity, rather than needing to reconsider the whole grid.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Number of islands after each add operation.

Visual Diagram

```
n=3, m=3, positions = [[0,0],[0,1],[1,2],[2,1]]
       After [0,0]: 1 island
       After [0,1]: 1 island (connected)
       After [1,2]: 2 islands
       After [2,1]: 2 islands? Actually [2,1] connects to [1,2]
```

Solution Approach

Concept: DSU with dynamic additions.

Algorithm

```
Algorithm NUM_ISLANDS_II(n, m, positions):
           grid = new int[n][m]
           dsu = DSU(n*m)
           directions = [[1,0],[-1,0],[0,1],[0,-1]]
           result = []
           count = 0

           For each position (x, y):
               If grid[x][y] == 1:
                   result.add(count)
                   continue
               grid[x][y] = 1
               count++
               index = x*m + y
               For each direction:
                   nx, ny = x+dx, y+dy
                   If valid and grid[nx][ny] == 1:
                       neighbor = nx*m + ny
                       If dsu.find(index) != dsu.find(neighbor):
                            dsu.union(index, neighbor)
                            count--
               result.add(count)
           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> numIslands2(int n, int m, int[][] positions) {
               int[][] grid = new int[n][m];
               DSU dsu = new DSU(n * m);
               List<Integer> result = new ArrayList<>();
               int count = 0;
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 for (int[] pos : positions) {
                     int x = pos[0], y = pos[1];
                     if (grid[x][y] == 1) {
                         result.add(count);
                         continue;
                     }
                     grid[x][y] = 1;
                     count++;
                     int index = x * m + y;

                   for (int[] dir : dirs) {
```

```
                        int nx = x + dir[0];
                        int ny = y + dir[1];
                        if (nx >= 0 && nx < n && ny >= 0 && ny < m && grid[nx][ny]
== 1) {
                            int neighbor = nx * m + ny;
                            if (dsu.find(index) != dsu.find(neighbor)) {
                                dsu.union(index, neighbor);
                                count--;
                            }
                        }
                    }
                    result.add(count);
                }
                return result;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n * α(n)) | Space Complexity: O(n)

348. Making a Large Island

Understanding the Problem NEW

Given a binary grid, you're allowed to change *at most one* 0 to a 1; find the size of the largest possible island achievable by choosing the best possible cell to flip.

Brute Force Approach NEW

Try flipping every single 0 cell one at a time, and for each, recompute the resulting largest island size from scratch via a full grid traversal — O(cells) flips considered, each requiring O(cells) to re-evaluate, giving O(cells²) overall.

Why We Optimize NEW

First, run one pass identifying and **labeling every existing island** with a unique ID, also recording each island's size (using DFS/BFS, or equivalently Union-Find grouping connected land cells). Then, for every 0 cell, look at its up/down/left/right neighbours' island IDs — a flip at that cell would merge together *all distinct* islands touching it (plus the newly flipped cell itself), so the resulting size is `1 + sum of sizes of each distinct neighbouring island` (being careful not to double-count the same island if it touches the 0 cell from multiple sides). Precomputing island sizes once (O(cells)) and then evaluating every 0 cell's potential in O(1) (just checking up to 4 neighbours) brings the total down to O(cells), instead of O(cells²) from re-scanning the whole grid for every candidate flip.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Make largest island by flipping one 0 to 1.

Visual Diagram

```
Grid:
       1 0 1
       0 1 0
       1 0 1

       Max island after flipping one 0 = 5
```

```
       DSU on all 1s, then check each 0
```

Solution Approach

Concept: DSU on land cells.

Algorithm

```
Algorithm LARGEST_ISLAND(grid):
           n = grid.length
           dsu = DSU(n*n)

           // Union all adjacent 1s
           For each cell with grid[i][j] == 1:
               For each neighbor with 1:
                   union(index, neighbor)

           // For each 0, check unique adjacent islands
           max = max component size
           For each cell with grid[i][j] == 0:
               Set roots of adjacent 1s
               size = 1 + sum of component sizes
               max = max(max, size)
           Return max
```

Java Implementation

```
class Solution {
           public int largestIsland(int[][] grid) {
               int n = grid.length;
               DSU dsu = new DSU(n * n);
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                 // Union all adjacent 1s
                 for (int i = 0; i < n; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1) {
                             int index = i * n + j;
                             for (int[] dir : dirs) {
                                 int ni = i + dir[0];
                                 int nj = j + dir[1];
                                 if (ni >= 0 && ni < n && nj >= 0 && nj < n &&
grid[ni][nj] == 1) {
                                     dsu.union(index, ni * n + nj);
                                 }
                             }
                         }
                     }
                 }
```

```
                // Get component sizes
                Map<Integer, Integer> sizeMap = new HashMap<>();
                for (int i = 0; i < n; i++) {
                    for (int j = 0; j < n; j++) {
                        if (grid[i][j] == 1) {
                            int root = dsu.find(i * n + j);
                            sizeMap.put(root, sizeMap.getOrDefault(root, 0) + 1);
                        }
                    }
                }

                int max = 0;
                for (int size : sizeMap.values()) {
                    max = Math.max(max, size);
                }

                // Try flipping each 0
                for (int i = 0; i < n; i++) {
                    for (int j = 0; j < n; j++) {
                        if (grid[i][j] == 0) {
                            Set<Integer> roots = new HashSet<>();
                            for (int[] dir : dirs) {
                                int ni = i + dir[0];
                                int nj = j + dir[1];
                                if (ni >= 0 && ni < n && nj >= 0 && nj < n &&
grid[ni][nj] == 1) {
                                    roots.add(dsu.find(ni * n + nj));
                                }
                            }
                            int size = 1;
                            for (int root : roots) {
                                size += sizeMap.get(root);
                            }
                            max = Math.max(max, size);
                        }
                    }
                }
                return max;
            }
        }
```

Time & Space Complexity

Time Complexity: $O(n^2 \alpha(n))$ | Space Complexity: $O(n^2)$

349. Swim in Rising Water

Understanding the Problem NEW

Given a grid representing elevations, and rising water starting at time 0, find the minimum time at which you can travel from the top-left to the bottom-right, where you can only move to a cell once the current water level (time) is ≥ that cell's elevation.

Brute Force Approach NEW

Try every possible path and, for each, compute the minimum time needed (the maximum elevation along that path) — exponential path exploration.

Why We Optimize NEW

This is exactly Path with Minimum Effort (331) in a new costume: you need the path that minimizes the **maximum elevation encountered** along the way (not the sum of elevations) — solved with the same **modified Dijkstra's Algorithm** that tracks 'minimize the max value seen so far' instead of 'minimize the total sum,' using a priority queue to always expand the currently-best (lowest max-elevation-so-far) path first. Recognizing this as the same underlying 'minimize the bottleneck value along a path' problem as 331, just with a different real-world story (water rising vs. hiking effort), avoids needing to design a new algorithm from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum time to reach bottom-right.

Visual Diagram

```
Grid:
       [0,2]
       [1,3]
       Time = 3

       Dijkstra with threshold:
       Start with time = grid[0][0]
```

```
       PQ by time
       Visit neighbors if time >= grid[neighbor]
```

Solution Approach

Concept: Dijkstra with elevation condition.

Algorithm

```
Algorithm SWIM_IN_WATER(grid):
           n = grid.length
           visited = boolean[n][n]
           pq = PriorityQueue((time, x, y))
           pq.add((grid[0][0], 0, 0))

            While pq not empty:
                (time, x, y) = pq.poll()
                If x == n-1 && y == n-1: return time
                For each neighbor:
                    newTime = Math.max(time, grid[nx][ny])
                    If !visited[nx][ny]:
                        visited[nx][ny] = true
                        pq.add((newTime, nx, ny))
            Return -1
```

Java Implementation

```
class Solution {
           public int swimInWater(int[][] grid) {
               int n = grid.length;
               boolean[][] visited = new boolean[n][n];
               PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] -
b[0]);
               pq.add(new int[]{grid[0][0], 0, 0});
               visited[0][0] = true;
               int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};

                  while (!pq.isEmpty()) {
                      int[] curr = pq.poll();
                      int time = curr[0], x = curr[1], y = curr[2];
                      if (x == n-1 && y == n-1) return time;

                     for (int[] dir : dirs) {
                         int nx = x + dir[0];
                         int ny = y + dir[1];
                         if (nx >= 0 && nx < n && ny >= 0 && ny < n && !visited[nx]
[ny]) {
                             visited[nx][ny] = true;
                             pq.add(new int[]{Math.max(time, grid[nx][ny]), nx,
ny});
```

```
                        }

                    }
                }
                return -1;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n² log n) | Space Complexity: O(n²)

350. Bridges in Graph

Understanding the Problem NEW

Find all 'bridges' in an undirected graph — edges whose removal would disconnect the graph (split it into more components than it currently has).

Brute Force Approach NEW

For every single edge, temporarily remove it and run a full connected-components check to see if the graph became more disconnected — O(E) edges to check, each requiring an O(V+E) traversal, giving O(E×(V+E)) overall.

Why We Optimize NEW

Use a single DFS traversal tracking two values per node: its **discovery time** (when it was first visited) and its **low-link value** (the earliest discovery time reachable from it, including through back-edges to ancestors) — an edge (u, v) is a bridge specifically if `low[v] > discovery[u]`, meaning v (and everything below it in the DFS tree) has *no other way* back to u or anything above u except through this exact edge. This computes discovery and low-link values for every node in a single O(V+E) DFS pass, identifying all bridges at once, instead of the O(E×(V+E)) cost of testing every edge's removal individually — this discovery-time/low-link technique is the foundation for both this problem and Articulation Points (351).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all bridges in an undirected graph.

Visual Diagram

```
Graph:
       0 --- 1
       |     |
       3 --- 2

       Bridge: 1-2? Wait, 1-2 is not a bridge
       Bridges: 0-3? Actually all edges are bridges? Let's check
       0-1: Removing disconnects 0 from 1,2,3 → bridge
```

```
1-2: Removing disconnects 2 from 0,1,3 → bridge
2-3: Removing disconnects 3 from 0,1,2 → bridge
3-0: Removing disconnects 0 from 1,2,3 → bridge
All edges are bridges (tree)

With cycle:
0 --- 1
|     |
3 --- 2
No bridges (cycle)
```

Solution Approach

Concept: DFS with discovery and low times.

Algorithm

```
Algorithm BRIDGES(graph):
           disc = new int[V], low = new int[V]
           visited = boolean[V]
           time = 0
           bridges = []

           DFS(node, parent):
               visited[node] = true
               disc[node] = low[node] = ++time

                  For neighbor in graph[node]:
                      If !visited[neighbor]:
                          dfs(neighbor, node)
                          low[node] = min(low[node], low[neighbor])
                          If low[neighbor] > disc[node]:
                              bridges.add((node, neighbor))
                      Else If neighbor != parent:
                          low[node] = min(low[node], disc[neighbor])
```

Java Implementation

```
class Solution {
           private int time = 0;

           public List<int[]> findBridges(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               for (int i = 0; i < n; i++) {

                  graph.add(new ArrayList<>());
              }
              for (int[] edge : edges) {
                  graph.get(edge[0]).add(edge[1]);
                  graph.get(edge[1]).add(edge[0]);
```

```
            }

            int[] disc = new int[n];
            int[] low = new int[n];
            boolean[] visited = new boolean[n];
            List<int[]> bridges = new ArrayList<>();

            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    dfs(i, -1, graph, disc, low, visited, bridges);
                }
            }
            return bridges;
        }

        private void dfs(int node, int parent, List<List<Integer>> graph,
                         int[] disc, int[] low, boolean[] visited, List<int[]>
bridges) {
            visited[node] = true;
            disc[node] = low[node] = ++time;

            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs(neighbor, node, graph, disc, low, visited, bridges);
                    low[node] = Math.min(low[node], low[neighbor]);
                    if (low[neighbor] > disc[node]) {
                        bridges.add(new int[]{node, neighbor});
                    }
                } else if (neighbor != parent) {
                    low[node] = Math.min(low[node], disc[neighbor]);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

351. Articulation Point

Understanding the Problem NEW

Find all 'articulation points' (cut vertices) in an undirected graph — nodes whose removal would increase the number of connected components (splitting the graph apart).

Brute Force Approach NEW

For every single node, temporarily remove it (and its edges) and check if the graph becomes more disconnected — O(V) nodes to check, each requiring an O(V+E) traversal.

Why We Optimize NEW

Extend the exact same discovery-time/low-link DFS technique from Bridges (350): a non-root node u is an articulation point if it has a child v in the DFS tree where `low[v] ≥ discovery[u]` (meaning v's subtree has no back-edge escaping past u, so removing u would strand it) — with a special-case rule for the DFS root (which is an articulation point specifically if it has more than one child in the DFS tree, since those children's subtrees can only be connected to each other through the root). This single O(V+E) DFS pass, essentially the same low-link machinery as Bridges but with a slightly different comparison condition (≥ instead of >, plus the root special case), finds every articulation point at once.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find articulation points (cut vertices).

Visual Diagram

```
Graph:
        0 --- 1
        |     |
        3 --- 2

        Articulation points: 1? Actually removing 1 disconnects 2 from 0,3
        Articulation points: 0,1,2,3 (tree)
```

```
        With cycle:
        0 --- 1
        |     |
        3 --- 2
        No articulation points
```

Solution Approach

Concept: DFS with discovery and low times.

Algorithm

```
Algorithm ARTICULATION_POINTS(graph):
            disc = new int[V], low = new int[V]
            visited = boolean[V]
            isArticulation = boolean[V]
            time = 0
            childCount = 0

            DFS(node, parent):
                visited[node] = true
                disc[node] = low[node] = ++time
                childCount = 0

                   For neighbor in graph[node]:
                       If !visited[neighbor]:
                           childCount++
                           dfs(neighbor, node)
                           low[node] = min(low[node], low[neighbor])
                           If parent == -1 && childCount > 1:
                               isArticulation[node] = true
                           If parent != -1 && low[neighbor] >= disc[node]:
                               isArticulation[node] = true
                       Else If neighbor != parent:
                           low[node] = min(low[node], disc[neighbor])
```

Java Implementation

```
class Solution {
            private int time = 0;

            public List<Integer> articulationPoints(int n, int[][] edges) {
                List<List<Integer>> graph = new ArrayList<>();
                for (int i = 0; i < n; i++) {
                    graph.add(new ArrayList<>());
                }
                for (int[] edge : edges) {

                  graph.get(edge[0]).add(edge[1]);
                  graph.get(edge[1]).add(edge[0]);
```

```
            }

            int[] disc = new int[n];
            int[] low = new int[n];
            boolean[] visited = new boolean[n];
            boolean[] isArticulation = new boolean[n];

            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    dfs(i, -1, graph, disc, low, visited, isArticulation);
                }
            }

            List<Integer> result = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                if (isArticulation[i]) result.add(i);
            }
            return result;
        }

        private void dfs(int node, int parent, List<List<Integer>> graph,
                         int[] disc, int[] low, boolean[] visited, boolean[]
isArticulation) {
            visited[node] = true;
            disc[node] = low[node] = ++time;
            int children = 0;

            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    children++;
                    dfs(neighbor, node, graph, disc, low, visited,
isArticulation);
                    low[node] = Math.min(low[node], low[neighbor]);

                    if (parent == -1 && children > 1) {
                        isArticulation[node] = true;
                    }
                    if (parent != -1 && low[neighbor] >= disc[node]) {
                        isArticulation[node] = true;
                    }
                } else if (neighbor != parent) {
                    low[node] = Math.min(low[node], disc[neighbor]);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V)

352. Kosaraju's Algorithm

Understanding the Problem NEW

Find all 'Strongly Connected Components' (SCCs) in a *directed* graph — maximal groups of nodes where every node in the group can reach every other node in the same group via directed paths (a much stricter notion of 'connected' than the undirected version, since direction matters).

Brute Force Approach NEW

For every pair of nodes, check if each can reach the other (two separate reachability checks per pair) — $O(V^2)$ pairs, each requiring an $O(V+E)$ traversal, giving $O(V^2 \times (V+E))$ overall.

Why We Optimize NEW

Kosaraju's Algorithm uses an elegant two-pass DFS strategy: first, do a DFS on the original graph, pushing each node onto a stack as it *finishes* (exactly like Topological Sort, problem 319); then, **reverse every edge** in the graph (transpose it); finally, pop nodes off the stack one at a time, and for each not-yet-visited node, run DFS on the *reversed* graph — each such DFS call discovers exactly one full SCC. This works because reversing the graph and processing in the finish-time-descending order (from the first pass) guarantees that a DFS in the second pass can't 'escape' into a different SCC without having already been captured within its own. Two $O(V+E)$ DFS passes total (plus the $O(V+E)$ edge-reversal step) gives $O(V+E)$ overall — dramatically better than the $O(V^2 \times (V+E))$ pairwise reachability check, by cleverly using the finishing-time ordering to guarantee each second-pass DFS stays contained within exactly one SCC.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find strongly connected components in directed graph.

Visual Diagram

```
Graph:
       0 → 1 → 2
       ■ ↓
       3 ← 4
```

```
Step 1: DFS order
DFS from 0: 0→1→2→4→3
Order: [0,1,2,4,3]

Step 2: Reverse graph
0 ← 1 ← 2
■ ↑
3 → 4

Step 3: DFS on reversed graph in reverse order
From 0: 0→3→4→1→2
SCC: [0,1,2,3,4]
```

Solution Approach

Concept: Two DFS passes.

Algorithm

```
Algorithm KOSARAJU(graph):
           // Pass 1: Ordering
           visited = boolean[V]
           stack = Stack()
           For i = 0 to V-1:
               If !visited[i]:
                   dfs1(i, visited, stack, graph)

           // Pass 2: Reverse graph
           revGraph = reverse(graph)
           visited = new boolean[V]
           sccs = []

           While stack not empty:
               node = stack.pop()
               If !visited[node]:
                   scc = []
                   dfs2(node, visited, scc, revGraph)
                   sccs.add(scc)
           Return sccs
```

Java Implementation

```
class Solution {
           public List<List<Integer>> kosaraju(int n, int[][] edges) {
               List<List<Integer>> graph = new ArrayList<>();
               List<List<Integer>> reverseGraph = new ArrayList<>();
               for (int i = 0; i < n; i++) {
                   graph.add(new ArrayList<>());
                   reverseGraph.add(new ArrayList<>());
```

```
            }

            for (int[] edge : edges) {
                graph.get(edge[0]).add(edge[1]);
                reverseGraph.get(edge[1]).add(edge[0]);
            }

            // Pass 1: Ordering
            boolean[] visited = new boolean[n];
            Stack<Integer> stack = new Stack<>();
            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    dfs1(i, graph, visited, stack);
                }
            }

            // Pass 2: SCC
            Arrays.fill(visited, false);
            List<List<Integer>> sccs = new ArrayList<>();
            while (!stack.isEmpty()) {
                int node = stack.pop();
                if (!visited[node]) {
                    List<Integer> scc = new ArrayList<>();
                    dfs2(node, reverseGraph, visited, scc);
                    sccs.add(scc);
                }
            }
            return sccs;
        }

        private void dfs1(int node, List<List<Integer>> graph, boolean[] visited,
Stack<Integer> st
            visited[node] = true;
            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs1(neighbor, graph, visited, stack);
                }
            }
            stack.push(node);
        }

        private void dfs2(int node, List<List<Integer>> graph, boolean[] visited,
List<Integer> scc
            visited[node] = true;
            scc.add(node);
            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    dfs2(neighbor, graph, visited, scc);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(V+E) | Space Complexity: O(V+E)

Graph Summary

Problem	Type	Time	Space	Key Technique
BFS/DFS	Learning	O(V+E)	O(V)	Queue/Stack
Provinces	BFS/DFS	$O(n^2)$	O(n)	DFS on matrix
Rotten Oranges	BFS/DFS	O(m*n)	O(m*n)	Multi-source BFS
Flood Fill	BFS/DFS	O(m*n)	O(m*n)	DFS with color
Cycle Undirected	BFS/DFS	O(V+E)	O(V)	Parent DFS
0/1 Matrix	BFS/DFS	O(m*n)	O(m*n)	Multi-source BFS
Surrounded Regions	BFS/DFS	O(m*n)	O(m*n)	Boundary DFS
Number of Enclaves	BFS/DFS	O(m*n)	O(m*n)	Boundary DFS
Word Ladder	BFS/DFS	O(n*L*26)	O(n)	BFS transformation
Distinct Islands	BFS/DFS	O(m*n)	O(m*n)	Shape normalization
Bipartite	BFS/DFS	O(V+E)	O(V)	2-coloring
Cycle Directed	BFS/DFS	O(V+E)	O(V)	3-state DFS
Topological Sort	Topo	O(V+E)	O(V)	DFS/Kahn
Kahn's Algorithm	Topo	O(V+E)	O(V)	Indegree queue
Course Schedule	Topo	O(V+E)	O(V)	Cycle detection
Alien Dictionary	Topo	O(n*L)	O(V+E)	Graph + Topo
Shortest Path UG	SP	O(V+E)	O(V)	BFS
Shortest Path DAG	SP	O(V+E)	O(V)	Topo + relaxation
Dijkstra	SP	O((V+E)logV)	O(V)	Priority queue
Binary Maze	SP	O(m*n)	O(m*n)	BFS on grid
Minimum Effort	SP	O(m*n log(mn))	O(m*n)	Modified Dijkstra
Cheapest Flights	SP	O(k*E)	O(V)	Bellman-Ford
Network Delay	SP	O((V+E)logV)	O(V)	Dijkstra
Count Ways	SP	O((V+E)logV)	O(V)	Dijkstra + DP
Bellman-Ford	SP	O(V*E)	O(V)	Relaxation
Floyd Warshall	SP	$O(V^3)$	$O(V^2)$	All-pairs DP
Prim's	MST	O(E log V)	O(V+E)	Priority queue
DSU	MST	$O(\alpha(n))$	O(n)	Union-Find

Kruskal	MST	O(E log E)	O(V)	DSU on sorted edges
Make Connected	DSU	O(E α(n))	O(n)	DSU counting
Accounts Merge	DSU	O(n log n)	O(n)	DSU on emails
Islands II	DSU	O(n α(n))	O(n)	Dynamic DSU
Largest Island	DSU	O(n² α(n))	O(n²)	DSU + flipping
Swim in Water	DSU/Dijkstra	O(n² log n)	O(n²)	Dijkstra with elevation
Bridges	Advanced	O(V+E)	O(V)	DFS low/disc
Articulation Point	Advanced	O(V+E)	O(V)	DFS low/disc
Kosaraju	Advanced	O(V+E)	O(V+E)	Two-pass DFS

— End of enhanced Chapter 12: Graphs — remaining chapters continue in the full edition —